


**Asignatura: Diseño web y Móvil**
**Solemne 2**
**Nombre del Alumno: Emiliano Lillo**
**Fecha: 09-11-2026**

# Índice


**1. Caso, alcance y requisitos** **3**
1.1 Identificación 3
1.2 Alcance de esta evaluación 3
1.3 Requisitos del caso y brechas declaradas 3
**2. Módulo 1: Servicios derivados del frontend** **5**
2.1 Catálogo de servicios 5
2.2 Qué envía el navegador y qué calcula el servidor 6
2.3 Ejemplos de contratos (datos ficticios) 6
**3. Módulo 2: Clases y cardinalidades del dominio** **8**
3.1 Diagramas 8
3.2 Relaciones y cardinalidades 11
3.3 Decisiones de modelado 12
3.4 Correspondencia entre operaciones UML y servicios 12
3.5 Modelo interno y DTO 13
**4. Módulo 3: Sistemas e integraciones** **14**
4.1 Matriz de sistemas 14
4.2 Quién decide el estado oficial 14
4.3 Mapa de componentes y secuencias 15
4.4 Integraciones y consistencia 17
**5. Módulo 4: Stakeholders y permisos HTTP** **19**
5.1 Registro de stakeholders 19
5.2 Matriz de permisos por servicio 19
5.3 Verbos HTTP por recurso 20
**6. Módulo 6: Rutas y secretos de integración** **21**
6.1 Rutas expuestas a los clientes 21
6.2 Rutas privadas entre sistemas 21
6.3 Operaciones GraphQL 21
6.4 Inventario de secretos 22
**7. Módulo 7: Datos sensibles y secretos del cliente** **23**
7.1 Matriz de datos 23
7.2 Decisión sobre secretos del cliente 23
7.3 Recorrido de un dato restringido: el RUN 23
7.4 Casos de validación ejecutados contra el backend 24
**8. Módulo 8: Separación privada y agregaciones GraphQL** **25**
8.1 Mapa de clasificación de servicios 25
8.2 Schema GraphQL 25
8.3 Resolvers y composición 26
8.4 Consulta de prueba y respuesta 26
8.5 Matriz de autorización 27
8.6 Límites de la consulta 27
**9. Módulo 9: Analítica y comunicación gRPC** **29**
9.1 Matriz de necesidades 29

9.2 ADR-001: Uso de gRPC y de un componente de analítica 29
9.3 Prueba de carga 29
9.4 Resultados medidos 30
**10. Módulo 10: Implementación e integración verificable 31**
10.1 Tecnologías y estructura 31
10.2 Persistencia y datos de prueba 31
10.3 Instrucciones de inicio, pruebas y detención 31
10.4 Matriz de trazabilidad 32
10.5 Resultado de la ejecución completa de la colección 32
10.6 Pruebas de integración y de fallos 33
10.7 Limitaciones declaradas 33
**11. Conclusiones 34**
**12. Referencias 35**
**Anexo A. Contratos completos (S01 a S39) 36**
**Anexo B. Código fuente y colección de Postman 37**
**Anexo C. Diagramas en formato editable 38**

# 1. Caso, alcance y requisitos

## 1.1 Identificación

| Campo | Detalle |
|---|---|
| Caso elegido | Caso 21: Bowling Health (tienda de comida saludable: bowls, wraps y ensaladas, con pedidos, pago y despacho). |
| Integrantes | Emiliano Lillo |
| Versión del frontend | Diseño/mock (pantallas P01 a P12). No se integra con el backend: es solo la fuente de requisitos. |
| Producto de esta entrega | Backend ejecutable (4 microservicios privados + 1 servicio de agregación GraphQL), código fuente, informe y pruebas ejecutadas en Postman. |

## 1.2 Alcance de esta evaluación

Esta evaluación corresponde únicamente al backend. Se acordó con el docente que los eventos de cada pantalla se simulan en Postman, con los datos que enviaría el frontend, y que ni el frontend responsivo ni el API Gateway forman parte de esta entrega (se evalúan por separado). Por ello cada microservicio se expone directamente en su puerto y es responsable de autenticar y autorizar sus propias peticiones.

El alcance de esta entrega fue acordado previamente con el docente, excluyendo el API Gateway y la integración con el frontend responsivo, los cuales son evaluados en instancias separadas.

## 1.3 Requisitos del caso y brechas declaradas

Las pantallas P01 a P04 existen en el diseño; las pantallas P05 a P12 son brechas declaradas. El backend implementa el 100% de las operaciones que todas las pantallas necesitan.

| Pantalla | Nombre | Estado | Acciones que requiere | Servicios |
|---|---|---|---|---|
| P01 | Menú / catálogo | Existe | Cargar productos, filtrar por categoría, ver detalle | S01, S02 |
| P02 | Promociones | Existe (sección vacía) | Mostrar promociones vigentes | S03 |
| P03 | Carrito | Existe (solo en el navegador) | Agregar y quitar productos. «Confirmar pedido» es una brecha | S21 |
| P04 | Iniciar sesión | Existe (simulado) | Iniciar y cerrar sesión, renovar sesión | S07, S08, S09 |
| P05 | Registro y verificación | Brecha | Crear cuenta, verificar correo, reenviar verificación | S04, S05, S06 |
| P06 | Confirmar pedido y pago | Brecha | Crear pedido, iniciar pago, ver estado del pago | S21, S25, S25b, S26 (sistema) |
| P07 | Mis pedidos y detalle | Brecha | Listar, ver detalle y anular pedidos | S22, S23, S24, S39 |
| P08 | Mi perfil y direcciones | Brecha | Ver y editar perfil, administrar direcciones | S10 a S14 |
| P09 | Administración | Brecha | Mantenedores de clientes, usuarios y productos | S15 a S20, S34 a S38 |
| P10 | Caja virtual | Brecha | Pedidos pagados, realizar venta y boleta | S27, S28, S29 |
| P11 | Despacho | Brecha | Cola de pedidos, vista imprimible, cambiar estado | S30, S31, S32 |
| P12 | Reportes | Brecha | Ventas por período | S33, S39 |

Equivalencia de nombres entre el frontend (español) y la API (inglés), para mantener la coherencia:

| Frontend | API | Frontend | API |
|---|---|---|---|
| categoria | category | imagen | imageUrl |

| Frontend | API | Frontend | API |
|---|---|---|---|
| nombre | name | cantidad | quantity |
| descripcion | description | precio | price (el servidor lo calcula; el navegador no lo envía) |

# 2. Módulo 1: Servicios derivados del frontend

Cada acción de las pantallas se convirtió en un servicio con identificador (S01 a S39, más S25b). Los servicios se reutilizan cuando varias pantallas ejecutan la misma operación. No se inventaron operaciones destructivas: las anulaciones y bajas son lógicas y conservan el historial.

## 2.1 Catálogo de servicios

Escala de clasificación: Público / Interno / Personal (PII) / Sensible (RUN, credenciales, tokens). La tabla completa con entrada, salida y reglas de cada servicio está en el Anexo A (Excel).

| ID | Pantalla | Verbo y ruta | Acceso | Clasificación de datos |
|---|---|---|---|---|
| S01 | P01 | GET /products | Anónimo | Público |
| S02 | P01 | GET /products/:id | Anónimo | Público |
| S03 | P02 | GET /promos | Anónimo | Público |
| S04 | P05 | POST /auth/register | Anónimo | Sensible (password, RUN, token). PII (email, nombre, teléfono) |
| S05 | P05 | POST /auth/verify-email | Anónimo | Sensible (token) |
| S06 | P05 | POST /auth/resend-verification | Anónimo | PII (email) |
| S07 | P04 | POST /auth/login | Anónimo | Sensible (credenciales, tokens) |
| S08 | Global | POST /auth/refresh | Anónimo (con refresh válido) | Sensible (tokens) |
| S09 | Global | POST /auth/logout | Cualquier rol autenticado | Sensible (tokens) |
| S10 | P08 | GET /clients/me | CLIENTE | Sensible (RUN), PII |
| S11 | P08 | PUT /clients/me | CLIENTE | PII |
| S12 | P08 | GET /clients/me/addresses | CLIENTE | PII |
| S13 | P08 | POST /clients/me/addresses | CLIENTE | PII |
| S14 | P08 | DELETE /clients/me/addresses/:id | CLIENTE | PII |
| S15 | P09 | GET /admin/clients | ADMIN | Sensible (RUN), PII |
| S16 | P09 | POST /admin/clients | ADMIN | Sensible (RUN), PII |
| S17 | P09 | PUT /admin/clients/:id | ADMIN | Sensible (RUN), PII |
| S18 | P09 | GET /admin/users | ADMIN | PII, Interno |
| S19 | P09 | POST /admin/users | ADMIN | Sensible (credenciales), PII |
| S20 | P09 | PUT /admin/users/:id | ADMIN | Interno |
| S21 | P03 → P06 | POST /orders | CLIENTE | Interno/financiero, PII (dirección) |
| S22 | P07 | GET /orders | CLIENTE | Interno/financiero |
| S23 | P07 | GET /orders/:id | CLIENTE (dueño) | Interno/financiero, PII |
| S24 | P07 | POST /orders/:id/cancellation | CLIENTE (dueño) o ADMIN | Interno |
| S25 | P06 | POST /payments | CLIENTE (dueño del pedido) | Interno/financiero |
| S25b | P06 | GET /payments/:orderId | CLIENTE (dueño) | Interno/financiero |
| S26 | Sistema | POST /webhooks/payments | Sistema (firma HMAC) | Interno/financiero. Secreto: Sensible |
| S27 | P10 | GET /cashier/orders | CAJERO | Interno/financiero, PII |
| S28 | P10 | POST /sales | CAJERO | Interno/financiero |
| S29 | P10 | GET /sales/:id/receipt | CAJERO, ADMIN, CLIENTE (dueño de la venta) | Interno/financiero, Sensible (RUN) |
| S30 | P11 | GET /dispatch/orders | DESPACHO | Interno, PII |
| S31 | P11 | GET /dispatch/orders/:id/print | DESPACHO | PII (dirección) |
| S32 | P11 | PUT /dispatch/orders/:id/status | DESPACHO | Interno |
| S33 | P12 | GET /reports/sales?desde&hasta | ADMIN | Interno/financiero |
| S34 | P09 | GET /admin/products | ADMIN | Interno |
| S35 | P09 | POST /admin/products | ADMIN | Interno |
| S36 | P09 | PUT /admin/products/:id | ADMIN | Interno |
| S37 | P09 | PUT /admin/products/:id/availability | ADMIN | Interno |
| S38 | P09 | DELETE /admin/products/:id | ADMIN | Interno |
| S39 | P07 / P12 | POST /graphql | CLIENTE (sus pedidos), ADMIN (reportes) | Interno/financiero, PII |

## 2.2 Qué envía el navegador y qué calcula el servidor

| Dato | Quién lo define | Cómo se evita la confianza en el cliente |
|---|---|---|
| Precio unitario, subtotal, despacho y total del pedido | Servidor (Orders con precios de Inventory) | El body de S21 no acepta total, unitPrice ni status: llegan como campos no permitidos y se responde 400 |
| Monto del pago | Servidor (total guardado en el pedido) | S25 solo recibe orderId; un campo amount se rechaza con 400 |
| Estado del pago y paso a PAID | Servidor (Payments, al validar el webhook) | Ningún rol puede mover un pedido a PAID por la API pública |
| Dueño del recurso (customerId) | Servidor (claim sub del JWT) | Nunca viene del body, de la URL ni de la query |
| Rol, estado de la cuenta y estado inicial del pedido | Servidor | Campos rechazados por la lista blanca de cada contrato |

## 2.3 Ejemplos de contratos (datos ficticios)

### S21 Crear pedido: request (sin total)

```
POST /orders
Authorization: Bearer <JWT del cliente>
Content-Type: application/json

{
  "addressId": "a3f1c2d4-5b6e-4f70-8a9b-0c1d2e3f4a5b",
  "items": [
    { "productId": 1, "quantity": 2 },
    { "productId": 4, "quantity": 1 }
  ],
  "notes": "Sin cebolla"
}
```

### S21 Response 201: calculado por el servidor

```
{
  "id": "o-20261104-000001",
  "status": "PENDING_PAYMENT",
  "subtotal": 16654,
  "deliveryFee": 1500,
  "total": 18154,
  "currency": "CLP",
  "items": [
    { "productId": 1, "name": "Bowl Mediterráneo", "quantity": 2, "unitPrice": 6291,
"lineTotal": 12582 },
    { "productId": 4, "name": "Wrap Vegetariano",  "quantity": 1, "unitPrice": 4072,
"lineTotal": 4072 }
  ],
  "shipping": { "alias": "Casa", "street": "Av. Providencia", "number": "1234", "commune":
"Providencia", "city": "Santiago" }
}
```

### Errores de S21

```
400  { "error": "VALIDATION_ERROR", "message": "Campos no permitidos: total", "fields":
["total"] }
404  { "error": "NOT_FOUND", "message": "Dirección no encontrada" }      // dirección de
otro cliente
422  { "error": "INSUFFICIENT_STOCK", "message": "Stock insuficiente para el producto 1",
"productId": 1 }
422  { "error": "PRODUCT_UNAVAILABLE", "message": "El producto 7 no está disponible",
"productId": 7 }
503  { "error": "DEPENDENCY_UNAVAILABLE", "message": "El servicio inventory no está
disponible" }
```

## S26 Webhook de pago (lo envía la pasarela, firmado)

```
POST /webhooks/payments
X-Timestamp: 1793720530
X-Signature: sha256=<HMAC-SHA256 de "timestamp.cuerpo">

{ "eventId": "evt-8812", "paymentId": "pay-5521", "status": "APPROVED", "amount": 18154,
"currency": "CLP" }

200 { "received": true }              // evento nuevo
200 { "received": true, "duplicate": true }   // mismo eventId: no se reprocesa
401 { "error": "INVALID_SIGNATURE" }
```

## S04 Registro y S10 Perfil (RUN protegido)

```
POST /auth/register
{ "email": "ana@correo.cl", "password": "Clave-Segura-2026!", "name": "Ana Rojas", "run":
"12.345.678-5", "phone": "+56912345678" }
202 { "message": "Si el correo es válido, recibirás instrucciones para verificar tu
cuenta." }

GET /clients/me
200 { "id": "…", "name": "Ana Rojas", "email": "ana@correo.cl", "phone": "+56912345678",
"runMasked": "12.345.***-5" }
```

## S05 Verificar correo (el token va en el body, nunca en la URL)

```
POST /auth/verify-email
{ "email": "ana@correo.cl", "token": "<token de 32 bytes>" }
200 { "status": "VERIFIED" }
400 { "error": "INVALID_TOKEN" }      // token inválido, usado o vencido
```

Los ejemplos de los servicios restantes (S01, S02, S03, S08, S09, S12, S13, S14, S15, S16, S17, S18, S19, S20, S22, S23, S24, S25b, S27, S28, S29, S30, S31, S32, S33, S34, S35, S36, S37 y S38) se encuentran detallados con sus respectivos contratos de entrada y salida en el **Anexo A (archivo Excel)** y pueden ser probados directamente en la **colección de Postman** adjunta."

# 3. Módulo 2: Clases y cardinalidades del dominio

El dominio se modeló con las entidades que necesitan los servicios del Módulo 1. Las relaciones entre microservicios se establecen por identificador, no por clave foránea, porque cada servicio es dueño exclusivo de su base de datos. Las dependencias UML entre servicios y entidades indican uso de un contrato, no relaciones entre registros.

## 3.1 Diagramas




*Figura 1. Diagrama de clases del servicio Clients (User, Client, Address, VerificationToken, RefreshToken).*

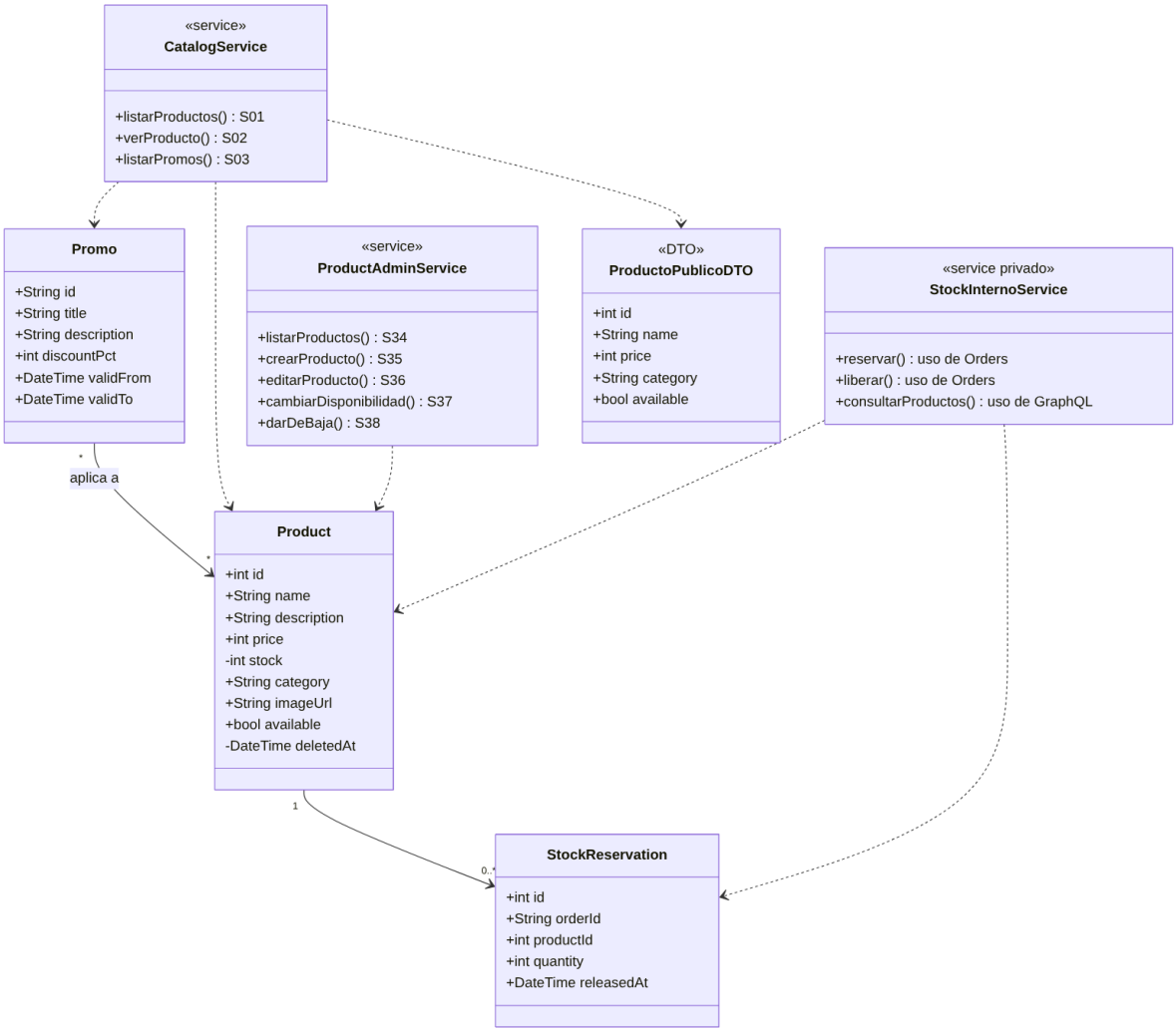



*Figura 2. Diagrama de clases del servicio Inventory (Product, Promo, StockReservation).*

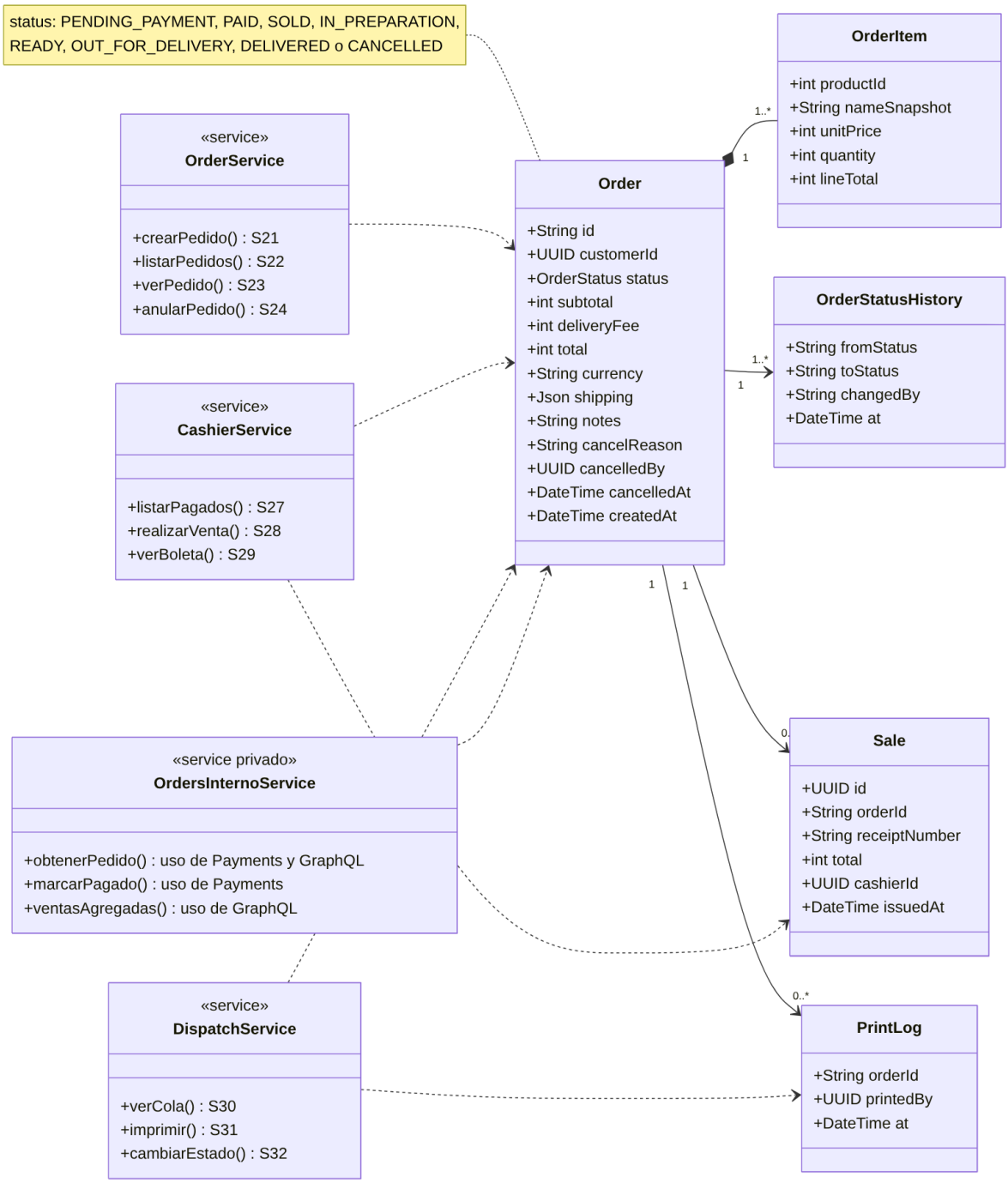



*Figura 3. Diagrama de clases del servicio Orders (Order, OrderItem, OrderStatusHistory, Sale, PrintLog).*

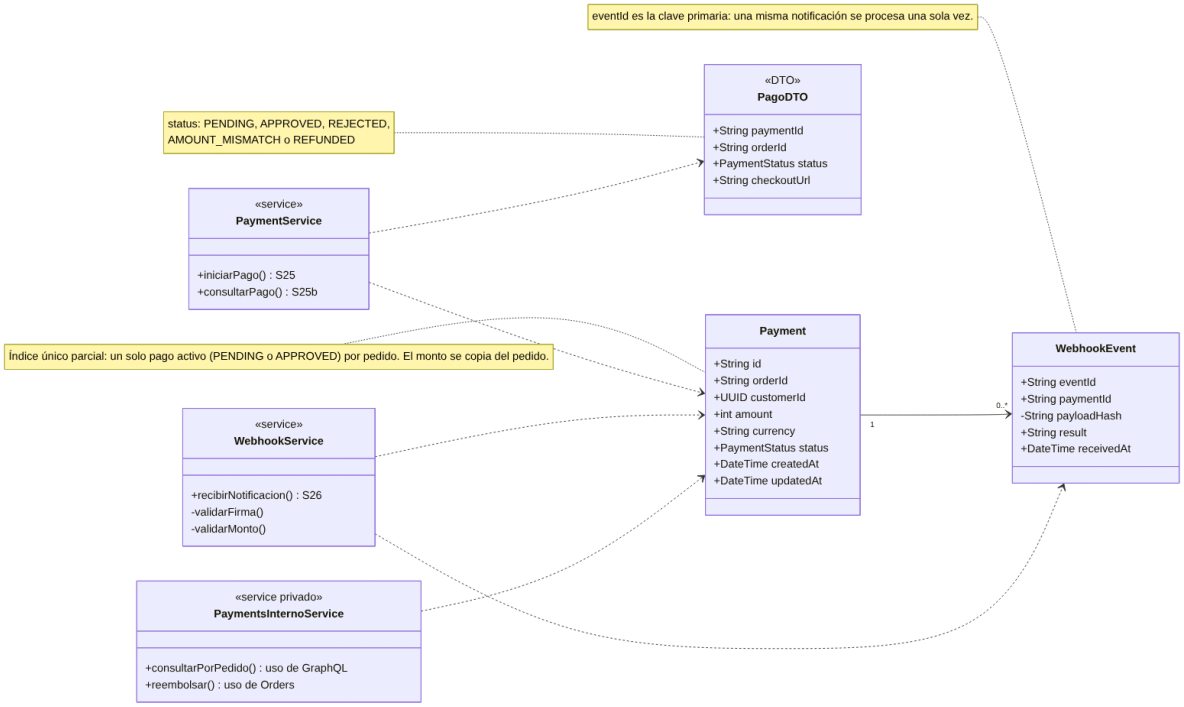



*Figura 4. Diagrama de clases del servicio Payments (Payment, WebhookEvent).*

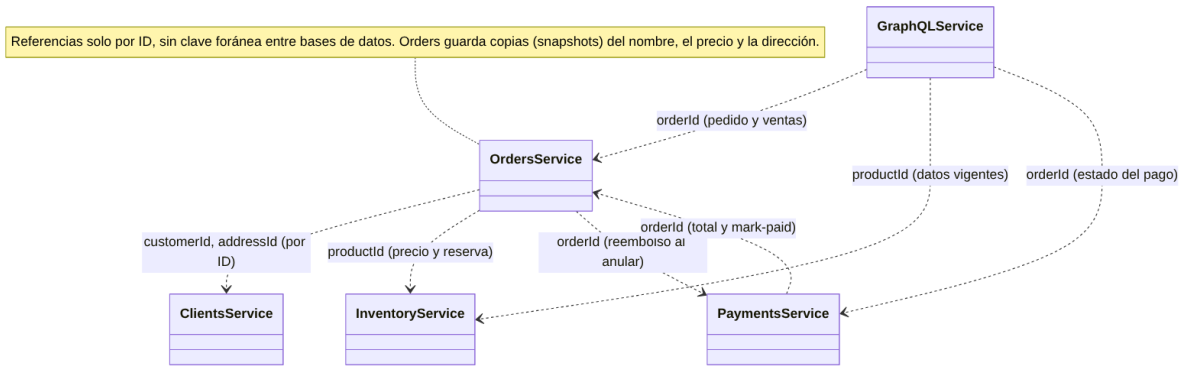



*Figura 5. Referencias entre servicios (solo por ID).*

### Leyenda de notación

Asociación: línea simple. Composición: rombo relleno (el detalle no existe sin su pedido). Dependencia: flecha punteada (uso de un contrato o de un ID de otro servicio). «service»: clase que expone operaciones del Módulo 1. «enumeration»: conjunto fijo de valores.

## 3.2 Relaciones y cardinalidades

| Origen | Destino | Mult. origen | Mult. destino | Regla de negocio que la respalda |
|---|---|---|---|---|
| User | Client | 1 | 0..1 | Solo los usuarios con rol CLIENTE tienen perfil de cliente; los usuarios internos (ADMIN, CAJERO, DESPACHO) no. |
| Client | Address | 1 | 0..10 | Un cliente puede no tener direcciones y tiene máximo 10 (S13). |
| User | RefreshToken | 1 | 0..* | Un usuario puede tener varias sesiones; cada refresh token pertenece a un usuario. |
| User | VerificationToken | 1 | 0..* | Cada reenvío genera un token nuevo e invalida el anterior. |
| Order (composición) | OrderItem | 1 | 1..* | Un pedido confirmado contiene al menos un ítem (máx. 50); cada ítem pertenece a un solo pedido. |

| Origen | Destino | Mult. origen | Mult. destino | Regla de negocio que la respalda |
|---|---|---|---|---|
| Order | OrderStatusHistory | 1 | 1..* | Todo pedido registra al menos su creación; cada cambio de estado agrega una fila. |
| Order | Sale | 1 | 0..1 | Un pedido pagado se vende una sola vez (restricción única sobre order_id). |
| Order | PrintLog | 1 | 0..* | Cada impresión para cocina queda registrada con su autor. |
| Product | StockReservation | 1 | 0..* | Cada pedido aparta unidades de uno o más productos; se liberan si el pedido se anula. |
| Promo | Product | * | * | Una promoción aplica a varios productos y un producto puede estar en varias promociones. |
| Payment | WebhookEvent | 1 | 0..* | Un pago puede recibir varias notificaciones; cada eventId se procesa una sola vez. |
| Order → Payment (por ID) | Payment | 1 | 0..* | Un pedido puede tener varios intentos de pago, pero solo uno activo (índice único parcial). |
| Client → Order (por ID) | Order | 1 | 0..* | Un cliente puede no tener pedidos; todo pedido pertenece a un cliente (customerId). |

## 3.3 Decisiones de modelado

| Pregunta | Decisión | Justificación |
|---|---|---|
| ¿Carrito y pedido son clases distintas? | Sí. El carrito vive en el navegador (como en el frontend actual) y solo se envía al backend al confirmar. | El backend no persiste borradores: el pedido nace ya con productos, precios reales y estado PENDING_PAYMENT. Por eso un pedido tiene siempre 1..* ítems. |
| ¿Qué pasa con el historial si un producto deja de estar disponible? | El pedido guarda una copia (snapshot) del nombre, el precio y la dirección. | La baja lógica de un producto o el borrado de una dirección no altera pedidos ya creados. |
| ¿Qué atributos se conservan y cuáles salen? | Ver tabla 3.5 | El DTO de cada respuesta contiene solo lo necesario para esa pantalla. |

## 3.4 Correspondencia entre operaciones UML y servicios

| Clase «service» | Operaciones (ID del Módulo 1) |
|---|---|
| AuthService | registrar() S04, verificarCorreo() S05, reenviarVerificacion() S06, iniciarSesion() S07, renovarToken() S08, cerrarSesion() S09 |
| ClientService | verPerfil() S10, actualizarPerfil() S11, listarDirecciones() S12, agregarDireccion() S13, eliminarDireccion() S14 |
| ClientAdminService | buscarClientes() S15, registrarClientePresencial() S16, editarCliente() S17, listarUsuarios() S18, crearUsuario() S19, editarUsuario() S20 |
| CatalogService | listarProductos() S01, verProducto() S02, listarPromos() S03 |
| ProductAdminService | listarProductos() S34, crearProducto() S35, editarProducto() S36, cambiarDisponibilidad() S37, darDeBaja() S38 |
| OrderService | crearPedido() S21, listarPedidos() S22, verPedido() S23, anularPedido() S24 |
| CashierService | listarPagados() S27, realizarVenta() S28, verBoleta() S29 |
| DispatchService | verCola() S30, imprimir() S31, cambiarEstado() S32 |
| PaymentService | iniciarPago() S25, consultarPago() S25b |
| WebhookService | recibirNotificacion() S26 |
| AggregationService (GraphQL) | reporteVentas() S33, detallePedido() S39 |

## 3.5 Modelo interno y DTO

| Entidad | Atributos internos que no salen | DTO que se entrega |
|---|---|---|
| User / Client | passwordHash, failedAttempts, lockedUntil, runEncrypted, runBlindIndex | id, name, email, phone, runMasked |

| Entidad | Atributos internos que no salen | DTO que se entrega |
|---|---|---|
| Product | stock, deletedAt (solo ADMIN los ve) | id, name, description, price, category, imageUrl, available |
| Order | customerId (solo en consultas internas), idempotencia del proveedor | id, status, subtotal, deliveryFee, total, items, shipping |
| Payment | customerId, providerRef | paymentId, orderId, status, updatedAt |
| Sale | cashierId | receiptNumber, issuedAt, total, items, customer (nombre y RUN enmascarado) |

# 4. Módulo 3: Sistemas e integraciones

El sistema se divide en cuatro microservicios de negocio (inventory, orders, clients, payments) y un servicio de agregación de solo lectura. Cada dato tiene una sola autoridad y ningún servicio accede a la base de datos de otro. Las rutas /internal/* solo aceptan la identidad de servicio; un JWT de usuario no sirve en ellas.

## 4.1 Matriz de sistemas

| Sistema | Responsabilidad y datos que administra | Servicios | Dependencias y contrato | Credencial requerida | Respuesta ante fallos |
|---|---|---|---|---|---|
| clients (:3001) | Registro, sesión, perfil, direcciones y usuarios internos. Autoridad de identidad y RUN. | S04 a S20 | Ninguna. Expone GET /internal/clients/:id/addresses/:addressId y /internal/clients/summaries | Posee la clave privada JWT, la clave AES del RUN y la clave del índice ciego | Cuenta bloqueada 15 min tras 5 intentos; respuestas genéricas |
| inventory (:3002) | Catálogo, promos, precios y stock. Autoridad del precio real y de la disponibilidad. | S01 a S03, S34 a S38 | Ninguna. Expone POST /internal/stock/reserve y /release, GET /internal/products | Verifica JWT (ADMIN). Rutas internas con token de servicio | Reserva transaccional con bloqueo de filas: nunca stock negativo. 422 si no hay stock |
| orders (:3003) | Ciclo de vida del pedido, venta, boleta y despacho. Autoridad del estado del pedido. | S21 a S24, S27 a S32 | Consulta a clients (dirección, nombre) e inventory (precios y reserva); llama a payments (reembolso) | Verifica JWT. Token de servicio hacia los demás | Si inventory no responde: 503 sin crear el pedido. Si falla al guardar: libera el stock (compensación) |
| payments (:3004) | Inicio de pagos, webhook y estado del pago. Autoridad del estado del pago. | S25, S25b, S26 | Consulta a orders (total y dueño) y llama a POST /internal/orders/:id/mark-paid | Secreto HMAC del webhook; token de servicio hacia orders | Si orders no responde en el webhook: se revierte y se responde 503 para que la pasarela reintente |
| graphql (:3005) | Agregación de solo lectura (S39) y reporte de ventas (S33). No tiene base de datos. | S33, S39 | orders, inventory y payments por rutas /internal | Verifica JWT; token de servicio hacia los demás | Error parcial: si inventory o payments caen, ese campo es null y el resto se entrega |

## 4.2 Quién decide el estado oficial

| Recurso | Sistema que decide | Transiciones permitidas |
|---|---|---|
| Pedido | orders | PENDING_PAYMENT → PAID (solo vía payments), PAID → SOLD (cajero), SOLD → IN_PREPARATION → READY → OUT_FOR_DELIVERY → DELIVERED (despacho), anulación desde PENDING_PAYMENT, PAID o SOLD |
| Pago | payments | PENDING → APPROVED / REJECTED / AMOUNT_MISMATCH (solo vía webhook validado), APPROVED → REFUNDED (anulación) |
| Stock y precio | inventory | Reservado al crear el pedido, devuelto al anularlo |
| Cuenta de usuario | clients | PENDING_VERIFICATION → ACTIVE (verificación de correo) → DISABLED (administrador) |

## 4.3 Mapa de componentes y secuencias

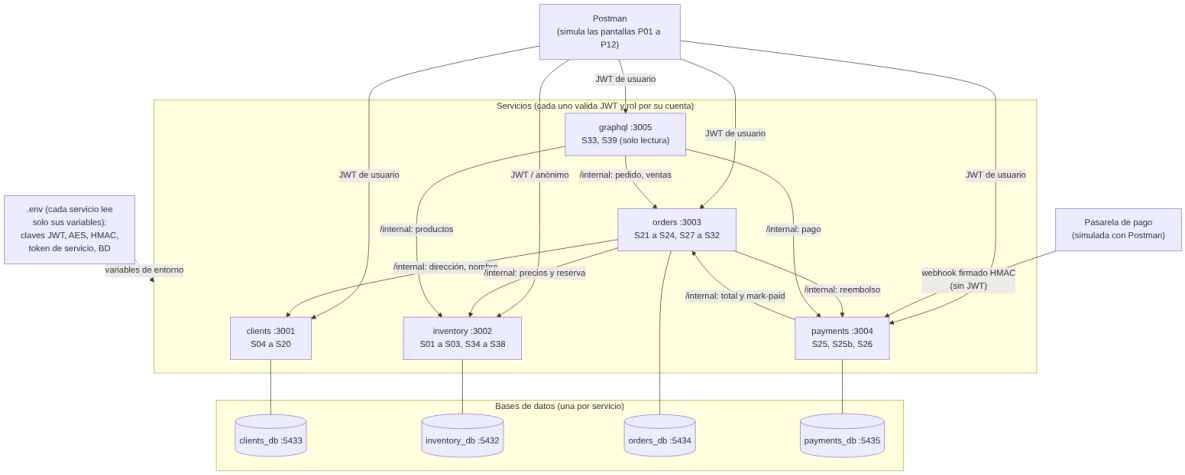



*Figura 6. Mapa de componentes: cinco servicios, cuatro bases de datos independientes y la pasarela de pago externa.*

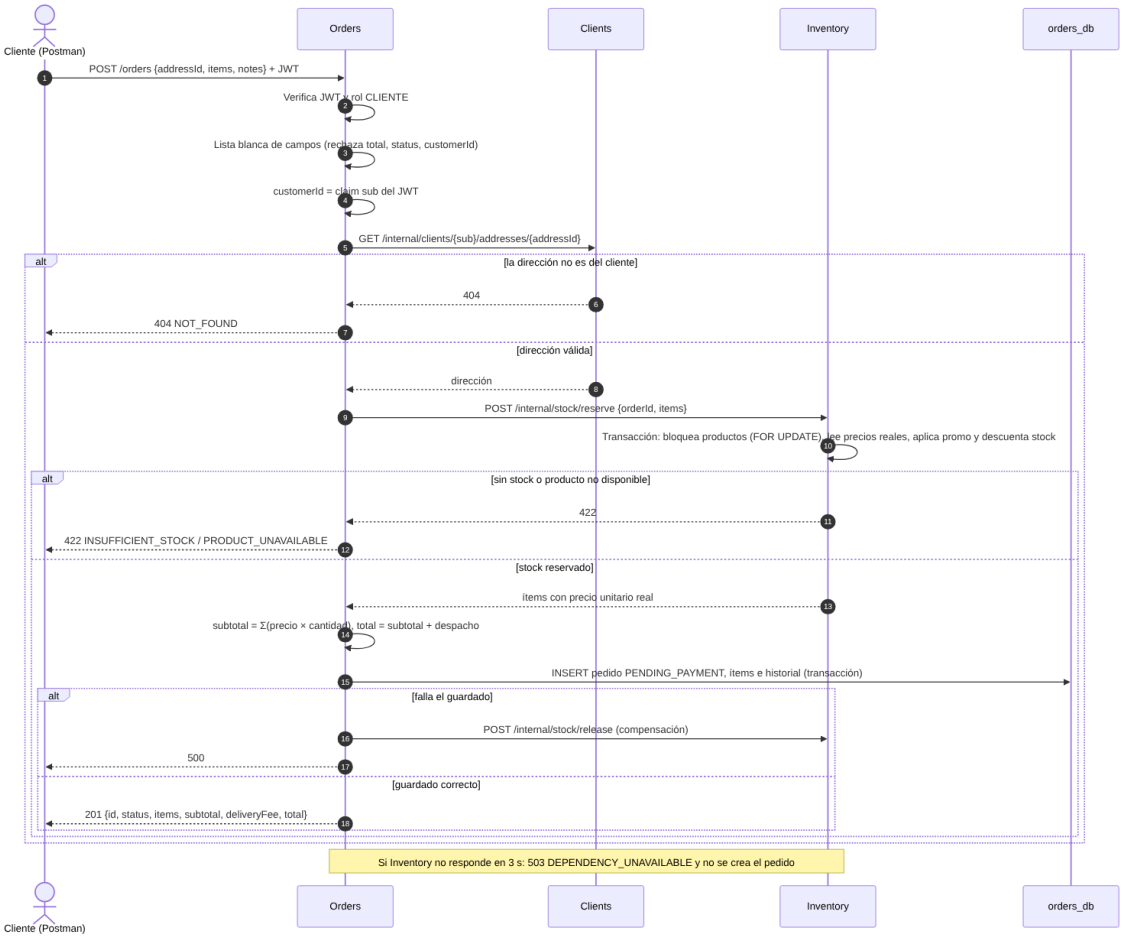



*Figura 7. Secuencia de S21 Crear pedido: Orders consulta a Clients (dirección) e Inventory (precios y reserva) y calcula el total.*

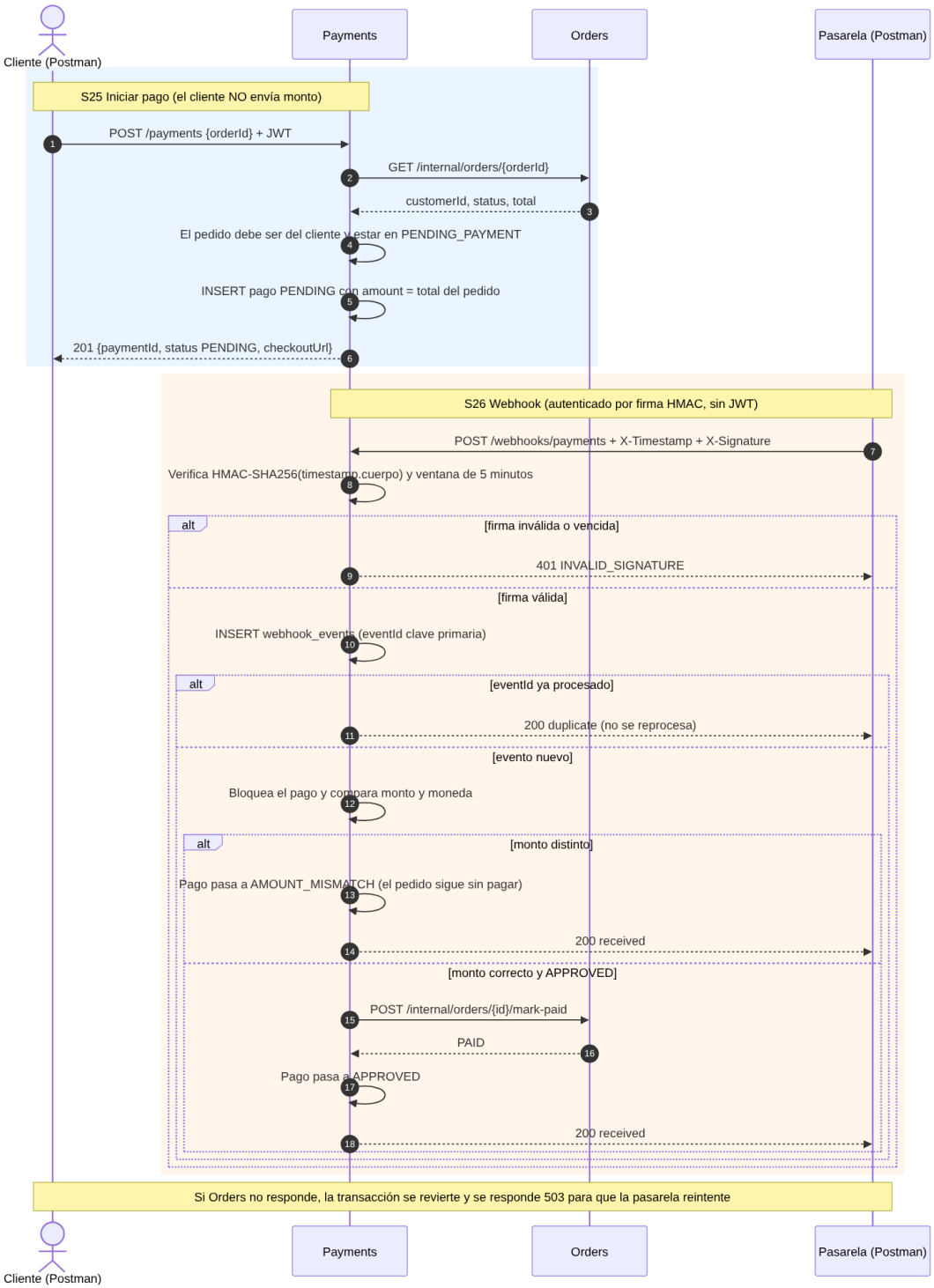



*Figura 8. Secuencia del pago: S25 y webhook S26 (firma, idempotencia, validación de monto, mark-paid).*

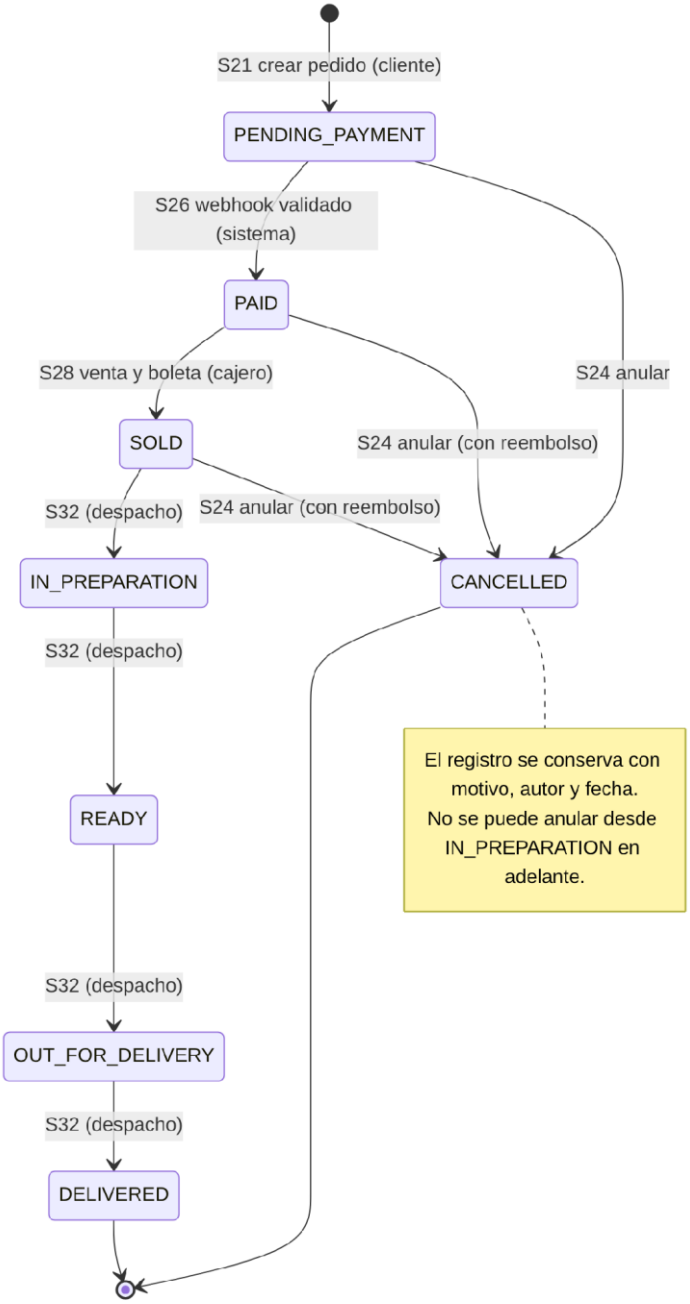



*Figura 9. Máquina de estados del pedido.*

## 4.4 Integraciones y consistencia

• Pasarela de pago: es un proveedor externo simulado. Postman actúa como pasarela y firma cada webhook con HMAC-SHA256 sobre «timestamp.cuerpo». El backend valida la firma en tiempo constante, rechaza notificaciones de más de 5 minutos y procesa cada eventId una sola vez.

• Si el monto o la moneda de la notificación no coinciden con el total guardado, el pago queda AMOUNT_MISMATCH y el pedido sigue sin pagar.

• Cada llamada entre servicios tiene un plazo máximo de 3 segundos y propaga el identificador de correlación.

• No se confirman unidades que ya no están disponibles: la reserva usa bloqueo de filas (SELECT … FOR UPDATE) dentro de una transacción.

# 5. Módulo 4: Stakeholders y permisos HTTP

## 5.1 Registro de stakeholders

| Stakeholder | Interés | Rol asociado | Uso de información |
|---|---|---|---|
| Visitante | Conocer el menú y las promociones | Anónimo (sin rol) | Catálogo y promos públicos |
| Cliente | Comprar, pagar y seguir sus pedidos | CLIENTE | Su perfil, direcciones, pedidos, pagos y boletas |
| Administrador | Mantener clientes, usuarios y productos; ver reportes | ADMIN | Datos del negocio; RUN siempre enmascarado |
| Cajero | Cobrar y emitir boletas | CAJERO | Pedidos pagados y boletas |
| Encargado de despacho | Preparar y entregar pedidos | DESPACHO | Cola de pedidos; dirección y notas (sin montos ni RUN) |
| Dueño del negocio | Supervisar las ventas | Sin permisos operativos | Recibe los reportes que genera el administrador (S33) |
| Pasarela de pago | Notificar el resultado de un pago | Identidad técnica (firma HMAC) | Solo puede invocar el webhook S26 |
| Servicios internos | Colaborar entre sí | Identidad técnica (token de servicio) | Solo rutas /internal/* |

## 5.2 Matriz de permisos por servicio

✓ permitido, — denegado. Sin token: 401. Rol sin permiso: 403. Recurso de otro usuario: 404 (no se revela su existencia). La verificación de rol y de propiedad ocurre en el servicio dueño del recurso; ocultar un botón en el frontend no sustituye este control.

| ID | Verbo y ruta | Anón. | CLIENTE | ADMIN | CAJERO | DESP. | Sistema | Condición |
|---|---|---|---|---|---|---|---|---|
| S01 | GET /products | ✓ | ✓ | ✓ | ✓ | ✓ | — | — |
| S02 | GET /products/:id | ✓ | ✓ | ✓ | ✓ | ✓ | — | — |
| S03 | GET /promos | ✓ | ✓ | ✓ | ✓ | ✓ | — | — |
| S04 | POST /auth/register | ✓ | ✓ | ✓ | ✓ | ✓ | — | — |
| S05 | POST /auth/verify-email | ✓ | ✓ | ✓ | ✓ | ✓ | — | — |
| S06 | POST /auth/resend-verification | ✓ | ✓ | ✓ | ✓ | ✓ | — | — |
| S07 | POST /auth/login | ✓ | ✓ | ✓ | ✓ | ✓ | — | — |
| S08 | POST /auth/refresh | ✓ | ✓ | ✓ | ✓ | ✓ | — | — |
| S09 | POST /auth/logout | — | ✓ | ✓ | ✓ | ✓ | — | — |
| S10 | GET /clients/me | — | ✓ | — | — | — | — | customerId = sub del JWT |
| S11 | PUT /clients/me | — | ✓ | — | — | — | — | customerId = sub del JWT |
| S12 | GET /clients/me/addresses | — | ✓ | — | — | — | — | customerId = sub |
| S13 | POST /clients/me/addresses | — | ✓ | — | — | — | — | customerId = sub |
| S14 | DELETE /clients/me/addresses/:id | — | ✓ | — | — | — | — | Dirección debe ser del sub; si no, 404 |
| S15 | GET /admin/clients | — | — | ✓ | — | — | — | — |
| S16 | POST /admin/clients | — | — | ✓ | — | — | — | — |
| S17 | PUT /admin/clients/:id | — | — | ✓ | — | — | — | — |
| S18 | GET /admin/users | — | — | ✓ | — | — | — | — |
| S19 | POST /admin/users | — | — | ✓ | — | — | — | — |
| S20 | PUT /admin/users/:id | — | — | ✓ | — | — | — | No se puede deshabilitar ni degradar al último ADMIN |
| S21 | POST /orders | — | ✓ solo propios | — | — | — | — | customerId = sub; addressId debe ser del sub |
| S22 | GET /orders | — | ✓ solo propios | — | — | — | — | Filtra por customerId = sub |
| S23 | GET /orders/:id | — | ✓ solo propios | — | — | — | — | order.customerId = sub; si no, 404 |
| S24 | POST /orders/:id/cancellation | — | ✓ solo propios | ✓ | — | — | — | CLIENTE solo si order.customerId = sub; ADMIN cualquiera |

| ID | Verbo y ruta | Anón. | CLIENTE | ADMIN | CAJERO | DESP. | Sistema | Condición |
|---|---|---|---|---|---|---|---|---|
| S25 | POST /payments | — | ✓ solo propios | — | — | — | — | El pedido debe ser del sub |
| S25b | GET /payments/:orderId | — | ✓ solo propios | — | — | — | — | El pedido debe ser del sub |
| S26 | POST /webhooks/payments | — | — | — | — | — | ✓ firma HMAC | Sin JWT: firma HMAC, timestamp e idempotencia |
| S27 | GET /cashier/orders | — | — | — | ✓ | — | — | — |
| S28 | POST /sales | — | — | — | ✓ | — | — | Solo pedidos PAID de clientes registrados |
| S29 | GET /sales/:id/receipt | — | ✓ solo propia | ✓ | ✓ | — | — | CLIENTE solo si la venta es de su pedido; si no, 404 |
| S30 | GET /dispatch/orders | — | — | — | — | ✓ | — | — |
| S31 | GET /dispatch/orders/:id/print | — | — | — | — | ✓ | — | — |
| S32 | PUT /dispatch/orders/:id/status | — | — | — | — | ✓ | — | Solo transiciones válidas de la máquina de estados |
| S33 | GET /reports/sales?desde&hasta | — | — | ✓ | — | — | — | — |
| S34 | GET /admin/products | — | — | ✓ | — | — | — | — |
| S35 | POST /admin/products | — | — | ✓ | — | — | — | — |
| S36 | PUT /admin/products/:id | — | — | ✓ | — | — | — | — |
| S37 | PUT /admin/products/:id/availability | — | — | ✓ | — | — | — | — |
| S38 | DELETE /admin/products/:id | — | — | ✓ | — | — | — | — |
| S39 | POST /graphql | — | ✓ solo propios | ✓ | — | — | — | CLIENTE solo orderDetail de sus pedidos; salesReport solo ADMIN |

## 5.3 Verbos HTTP por recurso

| Recurso | Visitante | CLIENTE | ADMIN | CAJERO | DESPACHO |
|---|---|---|---|---|---|
| Catálogo y promos (/products, /promos) | GET | GET | GET | GET | GET |
| Productos (/admin/products) | — | — | GET POST PUT DELETE (baja lógica) | — | — |
| Clientes y usuarios (/admin/clients, /admin/users) | — | — | GET POST PUT. DELETE: no aplica (se deshabilita) | — | — |
| Perfil y direcciones (/clients/me) | — | GET PUT; direcciones GET POST DELETE (solo propias) | — | — | — |
| Pedidos (/orders) | — | GET POST (solo propios); anular POST | Anular POST (cualquiera) | — | — |
| Pagos (/payments) | — | GET POST (solo sus pedidos) | — | — | — |
| Webhook (/webhooks/payments) | Solo con firma HMAC (identidad técnica de la pasarela): POST | | | | |
| Caja y boletas (/cashier, /sales) | — | GET boleta propia | GET boleta | GET POST | — |
| Despacho (/dispatch) | — | — | — | — | GET PUT |
| Reportes y GraphQL (/reports, /graphql) | — | POST /graphql (solo sus pedidos) | GET POST | — | — |
| Rutas internas (/internal/*) | Solo identidad de servicio | | | | |

Una anulación de pedido no borra el registro: el pedido queda en estado CANCELLED con el motivo, el autor y la fecha, y se conserva su historial de estados.

# 6. Módulo 6: Rutas y secretos de integración

*Sin API Gateway El Módulo 5 (Clases del API Gateway) no se incluye en este informe, ya que el API Gateway es evaluado de forma separada y no forma parte del alcance de esta entrega (ver Sección 1.2). Por lo tanto, la numeración de los módulos de este documento mantiene la referencia oficial de la guía para evitar confusiones, pero omite el desarrollo del Módulo 5.*

La ruta externa de cada servicio es la propia ruta del microservicio. Se documentan igualmente las rutas públicas, las rutas privadas entre sistemas y las credenciales que se usan en cada tramo.

## 6.1 Rutas expuestas a los clientes

| Servicio | Puerto | Rutas | Acceso | Controles |
|---|---|---|---|---|
| clients | 3001 | /auth/*, /clients/me/*, /admin/clients, /admin/users | Anónimo en /auth/*; JWT en el resto | Rate limit 60/min por IP; reenvío 3/hora; lista blanca de campos; bloqueo por intentos |
| inventory | 3002 | /products, /promos, /admin/products | Anónimo en catálogo; JWT ADMIN en /admin | Rate limit 100/min en catálogo; lista blanca |
| orders | 3003 | /orders, /cashier/orders, /sales, /dispatch/* | JWT (rol según ruta) | Propiedad del recurso, máquina de estados |
| payments | 3004 | /payments, /webhooks/payments | JWT CLIENTE; webhook con firma HMAC | Firma, anti-replay, idempotencia |
| graphql | 3005 | /graphql, /reports/sales | JWT | Profundidad máx. 5, campos máx. 60, rate limit 30/min |

## 6.2 Rutas privadas entre sistemas

| Ruta interna | Llama | Consume | Datos mínimos | Credencial |
|---|---|---|---|---|
| GET /internal/clients/:clientId/addresses/:addressId | orders | clients | Dirección del cliente (404 si no es suya) | SERVICE_TOKEN |
| GET /internal/clients/summaries?ids= | orders | clients | Nombre y RUN enmascarado | SERVICE_TOKEN |
| POST /internal/stock/reserve | orders | inventory | orderId e ítems; devuelve precios reales | SERVICE_TOKEN |
| POST /internal/stock/release | orders | inventory | orderId (idempotente) | SERVICE_TOKEN |
| GET /internal/products?ids= | graphql | inventory | Datos vigentes de productos | SERVICE_TOKEN |
| GET /internal/orders/:id | payments, graphql | orders | Pedido, dueño y total | SERVICE_TOKEN |
| POST /internal/orders/:id/mark-paid | payments | orders | Solo cambia PENDING_PAYMENT a PAID (idempotente) | SERVICE_TOKEN |
| GET /internal/sales/aggregate | graphql | orders | Ventas agregadas por día (sin datos personales) | SERVICE_TOKEN |
| GET /internal/payments/by-order/:id | graphql | payments | Estado del pago | SERVICE_TOKEN |
| POST /internal/payments/by-order/:id/refund | orders | payments | Reembolso de un pago aprobado (idempotente) | SERVICE_TOKEN |

## 6.3 Operaciones GraphQL

| Operación | Tipo | Campos autorizados | Resolver | Subsistemas consultados |
|---|---|---|---|---|
| orderDetail(id) | Query | CLIENTE: solo sus pedidos. ADMIN: cualquiera. Sin customerId ni run | Query.orderDetail, OrderLine.product, OrderDetail.payment | orders, inventory (por lote), payments |
| salesReport(desde, hasta) | Query | Solo ADMIN. Solo agregados | Query.salesReport | orders |

Métodos gRPC: no aplica. La decisión está fundamentada en el ADR del Módulo 9.

## 6.4 Inventario de secretos

| Referencia | Tipo | Emisor | Consumidor | Finalidad | Ubicación segura | Rotación y revocación |
|---|---|---|---|---|---|---|
| JWT_PRIVATE_KEY_B64 | Clave privada RS256 | Script de claves | clients | Firmar los access tokens | Archivo .env (fuera del repositorio) | Cada 90 días; si se filtra, rotar y revocar los refresh tokens |
| JWT_PUBLIC_KEY_B64 | Clave pública | Script de claves | Todos los servicios | Verificar la firma de los tokens | .env (no es secreta) | Junto con la privada |
| RUN_AES_KEY_B64 | Clave AES-256 | Script de claves | clients | Cifrar el RUN en la BD | .env | Anual, con recifrado de los RUN |
| RUN_BLINDINDEX_KEY_B64 | Clave HMAC | Script de claves | clients | Búsqueda y unicidad del RUN | .env | Anual, con reindexado |
| PAYMENT_WEBHOOK_SECRET | Secreto HMAC | Acordado con la pasarela | payments (valida) y pasarela (firma) | Autenticar el webhook | .env | Cada 90 días, aceptando ambos durante la transición |
| SERVICE_TOKEN | Token de servicio | Script de claves | Todos los servicios | Autenticar llamadas /internal/* | .env | Cada 90 días |
| *_DB_PASSWORD (4) | Credencial de BD | Script de claves | Un servicio cada una | Acceso a su propia base | .env | Cada 90 días |
| ADMIN_PASSWORD | Contraseña inicial | Script de claves | clients (solo al crear el primer ADMIN) | Primer administrador | .env | Cambiar tras el primer uso |

Decisión declarada: las credenciales de base de datos son distintas por servicio y las claves de RUN solo las posee clients. En cambio, el token de servicio es un único secreto compartido por todos los servicios; se aceptó por simplicidad, y la mejora prevista es un token por par de servicios con permisos mínimos (por ejemplo, payments solo podría invocar mark-paid). Ningún secreto se incluye en el código ni en el informe: viven en el archivo .env, excluido del repositorio.

# 7. Módulo 7: Datos sensibles y secretos del cliente

## 7.1 Matriz de datos

| Dato | Finalidad | Clasificación | Custodio | Roles autorizados | Protección | Retención |
|---|---|---|---|---|---|---|
| Contraseña | Autenticar al usuario | Sensible (credencial) | clients | Nadie la consulta | Hash Argon2id con sal; nunca se devuelve | Mientras exista la cuenta |
| RUN | Identificar al cliente (boleta, unicidad) | Sensible | clients | El cliente y ADMIN ven solo la versión enmascarada | AES-256-GCM en BD; índice ciego HMAC; enmascarado 12.345.***-5 | Mientras exista la cuenta; no hay borrado físico |
| Token de verificación de correo | Confirmar el correo | Sensible | clients | Nadie | Solo se guarda su hash SHA-256; un solo uso; vence en 24 h; viaja en el body | Expira a las 24 h |
| Refresh token | Mantener la sesión | Sensible | clients | Nadie | Solo hash; rotación; se revoca en logout y al cambiar rol o estado | 7 días |
| Access token (JWT) | Autorizar peticiones | Sensible | No se guarda | Portador | RS256, 15 minutos | 15 minutos |
| Correo, nombre, teléfono | Contactar y facturar | Personal (PII) | clients | Dueño y ADMIN | Acceso por propiedad o rol; mínimo necesario | Mientras exista la cuenta |
| Dirección de entrega | Entregar el pedido | Personal (PII) | clients; copia en orders | Dueño; DESPACHO solo la del pedido | No se incluye en el catálogo ni en listados | Con el pedido (copia histórica) |
| Pedidos, pagos, boletas | Operación y contabilidad | Interno / financiero | orders, payments | Dueño y roles operativos | El servidor calcula los montos; son inmutables tras crearse | Se conservan por trazabilidad (plazo a definir con el negocio) |
| Secreto del webhook | Autenticar a la pasarela | Sensible (secreto de integración) | payments | Nadie | Solo en .env; el navegador nunca lo recibe | Rotación cada 90 días |
| Datos de tarjeta | Pagar | No se almacenan | Pasarela externa | — | No pasan por el backend | No aplica |

## 7.2 Decisión sobre secretos del cliente

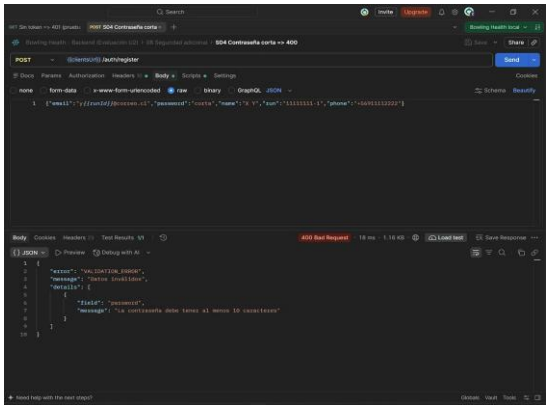
Sí custodiamos un secreto del cliente: su contraseña (como hash adaptativo con sal, no cifrado reversible) y los tokens de sesión y verificación (como hashes). No custodiamos claves de pago ni datos de tarjeta del cliente: la pasarela los recibe directamente. El RUN es un dato sensible, no una clave, y se protege con cifrado reversible porque debe poder mostrarse enmascarado. Ningún secreto se incluye en HTML, JavaScript, capturas ni ejemplos.

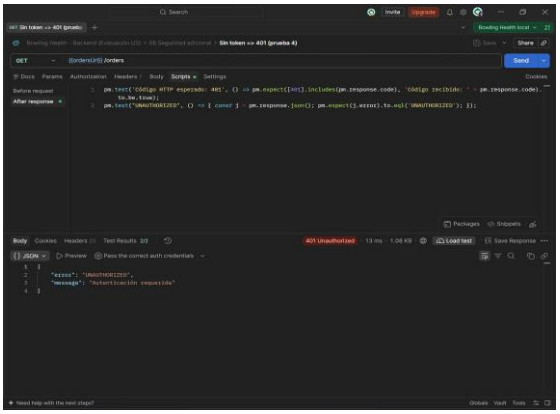
## 7.3 Recorrido de un dato restringido: el RUN

| Etapa | Qué ocurre |
|---|---|
| Captura | El cliente lo ingresa en el registro (S04) o lo registra un administrador (S16). Viaja en el body y, en producción, bajo TLS. |
| Validación | Se normaliza y se verifica el dígito verificador (módulo 11). Si es inválido: 400 INVALID_RUN. |
| Almacenamiento | Se cifra con AES-256-GCM (IV aleatorio por registro) y se guarda un índice ciego HMAC para búsqueda y unicidad. El texto claro no se guarda. |
| Uso | Se descifra solo en memoria para enmascararlo. Ninguna respuesta lo devuelve completo, ni siquiera al administrador ni en la boleta. Los logs no registran cuerpos. |
| Corrección | Solo el administrador (S17): se revalida, se recifra y se recalcula el índice. |
| Conservación o eliminación | No existe borrado físico: la baja de un cliente es lógica (estado DISABLED) para conservar el historial de pedidos y boletas (obligación contable). Sin embargo, ante una solicitud de eliminación por parte del titular (derecho de supresión), el administrador puede anonimizar los datos personales (nombre, correo, teléfono) manteniendo únicamente el RUN cifrado y el historial de compras para fines estadísticos y tributarios. |

## 7.4 Casos de validación ejecutados contra el backend

| # | Caso | Entrada e identidad | Resultado esperado | Componente que lo verifica | Resultado obtenido |
|---|---|---|---|---|---|
| 1 | Acceso anónimo al catálogo | GET /products sin token | 200, sin stock | inventory | 200 OK |
| 2 | Operación protegida sin sesión | GET /orders sin token | 401 | orders (middleware JWT) | 401 Unauthorized |
| 3 | Administración con rol sin permiso | Cliente: POST /admin/products | 403 | inventory (requireRole) | 403 Forbidden |
| 4 | Pedido ajeno | Cliente 2: GET /orders/:id de otro | 404 sin datos | orders (propiedad en la consulta) | 404 NOT_FOUND |
| 5 | Acceso directo al backend sin identidad | GET /internal/orders/:id sin token de servicio, y con JWT de ADMIN | 401 | orders (serviceAuth) | 401 Unauthorized |
| 6 | Confirmar pago con datos alterados | POST /payments con amount; webhook con firma inválida; webhook con monto distinto | 400; 401; AMOUNT_MISMATCH (el pedido sigue sin pagar) | payments | 400 Bad Request / 401 Invalid Signature / AMOUNT_MISMATCH |
| 7 | Total enviado por el navegador | POST /orders con total | 400 (campo no permitido) | orders (lista blanca) | 400 VALIDATION_ERROR |
| 8 | Webhook repetido | Mismo eventId dos veces | 200 duplicate, sin reprocesar | payments (idempotencia) | 200 (duplicate: true) |
| 9 | RUN protegido | GET /clients/me | runMasked con *** | clients | runMasked (con asteriscos) |
| 10 | Token de verificación en la URL | POST /auth/verify-email?token=… sin body | 400 | clients | 400 Bad Request |





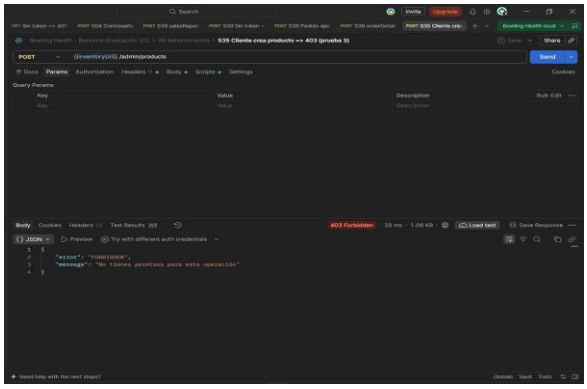


*Figura 10 Runner seguridad*

# 8. Módulo 8: Separación privada y agregaciones GraphQL

## 8.1 Mapa de clasificación de servicios

| Servicio | Naturaleza | Responsabilidad | Datos propios | Dependencias | Justificación de la separación |
|---|---|---|---|---|---|
| clients | Transaccional | Identidad, sesión, perfil, direcciones | users, clients, addresses, tokens | Ninguna | Concentra los datos más sensibles (RUN, credenciales); solo él posee sus claves |
| inventory | Transaccional | Catálogo, precios y stock | products, promos, reservas | Ninguna | Es la autoridad del precio y del stock; tiene un perfil de lectura alto y público |
| orders | Transaccional | Pedidos, venta y despacho | orders, items, sales, historial | clients, inventory, payments | Ciclo de vida completo del pedido con su máquina de estados |
| payments | Transaccional e integración externa | Pagos y webhook | payments, webhook_events | orders | Aísla el secreto del proveedor y las reglas de idempotencia |
| graphql | Agregación (solo lectura) | Combinar datos para una pantalla | Ninguno | orders, inventory, payments | Evita que el frontend llame a varios servicios y no accede a ninguna BD |
| Analítica y alta demanda | No implementados | — | — | — | Justificado en el ADR (Módulo 9) |

No se dividió un servicio por tabla ni por pantalla: caja y despacho viven en orders porque comparten el mismo pedido y su máquina de estados.

## 8.2 Schema GraphQL

```
# Servicio GraphQL (puerto 3005): solo lectura. No existe ningún campo "run" ni
credenciales.
scalar DateTime

enum OrderStatus {
  PENDING_PAYMENT
  PAID
  SOLD
  IN_PREPARATION
  READY
  OUT_FOR_DELIVERY
  DELIVERED
  CANCELLED
}

enum PaymentStatus {
  PENDING
  APPROVED
  REJECTED
  AMOUNT_MISMATCH
  REFUNDED
}

type Query {
  "Detalle de pedido: Orders + Inventory + Payments (pantalla P07). CLIENTE: solo los
suyos. ADMIN: cualquiera."
  orderDetail(id: ID!): OrderDetail!
  "Ventas por período (pantalla P12). Solo ADMIN."
  salesReport(desde: DateTime!, hasta: DateTime!): SalesReport!
}

type OrderDetail {
  id: ID!
  status: OrderStatus!
  createdAt: DateTime!
  subtotal: Int!
  deliveryFee: Int!
  total: Int!
  currency: String!
  shipping: ShippingInfo!
  items: [OrderLine!]!
  "Datos de Payments (null si Payments no responde: error parcial controlado)"
```

```
  payment: PaymentInfo
}

type ShippingInfo {
  street: String!
  number: String!
  commune: String!
  city: String!
  notes: String
}

type OrderLine {
  productId: ID!
  "Copia guardada en el pedido (Orders)"
  name: String!
  quantity: Int!
  unitPrice: Int!
  lineTotal: Int!
  "Datos vigentes del producto (Inventory); null si Inventory no responde"
  product: ProductLive
}

type ProductLive {
  id: ID!
  currentPrice: Int!
  available: Boolean!
  category: String
  imageUrl: String
}

type PaymentInfo {
  status: PaymentStatus!
  updatedAt: DateTime!
}

type SalesReport {
  desde: DateTime!
  hasta: DateTime!
  totalSales: Int!
  totalAmount: Int!
  byDay: [DaySales!]!
}

type DaySales {
  date: String!
  sales: Int!
  amount: Int!
}
```

## 8.3 Resolvers y composición

- orderDetail consulta primero a Orders y verifica la propiedad: si el pedido es de otro cliente responde NOT_FOUND, igual que si no existiera.
- OrderLine.product obtiene los datos vigentes en Inventory con un DataLoader que agrupa todas las líneas en una sola llamada (sin una llamada por ítem).
- OrderDetail.payment consulta a Payments solo si el cliente pidió ese campo.
- Si Inventory o Payments no responden, ese campo se entrega como null y el resto de la respuesta se mantiene (error parcial controlado).
- Las reglas transaccionales permanecen en cada servicio de negocio; el servicio GraphQL solo lee y no tiene acceso a ninguna base de datos.

## 8.4 Consulta de prueba y respuesta

```
query($id: ID!) {
  orderDetail(id: $id) {
    id status total
    shipping { street commune }
    items { name quantity unitPrice product { currentPrice available } }
    payment { status }
  }
}

variables: { "id": "o-20261104-000001" }


{ "data": { "orderDetail": {
  "id": "o-20261104-000001", "status": "DELIVERED", "total": 18154,
  "shipping": { "street": "Av. Providencia", "commune": "Providencia" },
  "items": [
    { "name": "Bowl Mediterráneo", "quantity": 2, "unitPrice": 6291,
      "product": { "currentPrice": 6990, "available": true } }
  ],
  "payment": { "status": "null" } } } } "El pedido aún no tiene un pago registrado, por eso
payment es null."
```

## 8.5 Matriz de autorización

| Operación o campo | CLIENTE | ADMIN | Otros roles | Regla |
|---|---|---|---|---|
| orderDetail | Solo sus pedidos | Cualquiera | FORBIDDEN | Propiedad verificada en el resolver |
| orderDetail.items.product | Sí | Sí | — | Datos públicos de Inventory |
| orderDetail.payment | Sí (su pedido) | Sí | — | Hereda la autorización del pedido |
| salesReport | FORBIDDEN | Sí | FORBIDDEN | Solo agregados, sin datos personales |
| customerId, run (campos inexistentes) | Error de validación | Error de validación | — | No existen en el schema: no se pueden solicitar |
| Sin token | UNAUTHENTICATED | | | Todas las operaciones exigen JWT |

## 8.6 Límites de la consulta

Profundidad máxima de 5 niveles, máximo 60 campos por consulta, 30 peticiones por minuto, cuerpo máximo de 100 KB, introspección desactivada cuando NODE_ENV=production y errores enmascarados. Cada servicio privado además rechaza llamadas sin la identidad de servicio.

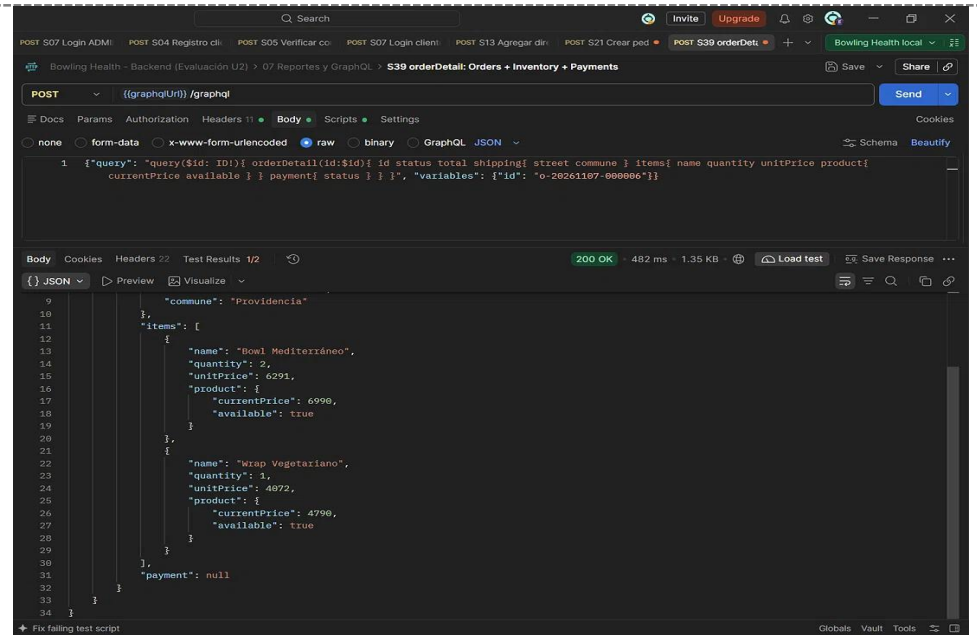



*Figura 11. Consulta orderDetail ejecutada en Postman por el cliente dueño (respuesta combinada).*

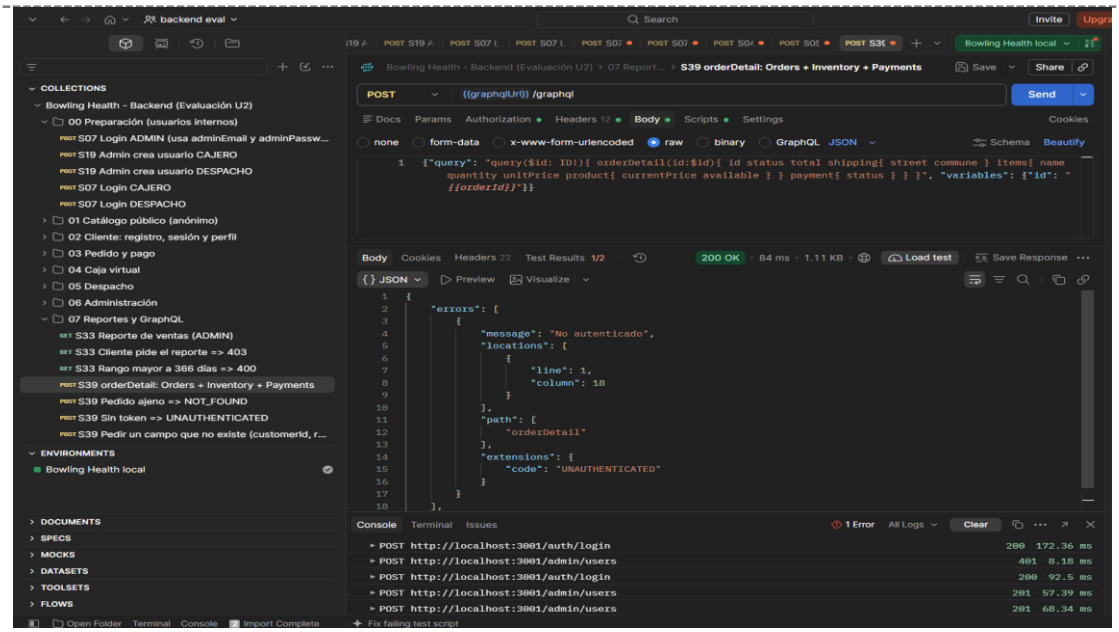



*Figura 12. Misma consulta con otro cliente: UNAUTHENTICATED, sin datos.*

# 9. Módulo 9: Analítica y comunicación gRPC

## 9.1 Matriz de necesidades

Los valores son proyecciones propuestas por el equipo para un local pequeño; no son mediciones. Las mediciones reales se registran en el apartado 9.4.

| Aspecto | Valor | Tipo | Consecuencia para el diseño |
|---|---|---|---|
| Pedidos por día | 150 (proyección) | Proyección | Volumen bajo: no requiere procesamiento masivo |
| Pico de concurrencia | 20 peticiones por segundo en catálogo | Proyección | Una instancia por servicio basta; se verifica en la prueba de carga |
| Llamadas internas por pedido | 2 a 3 (clients, inventory, orders↔payments) | Medido en el diseño | El costo de REST es despreciable frente a gRPC |
| Tamaño de mensajes | Menos de 5 KB | Proyección | No se necesita streaming |
| Reportes | Bajo demanda, rango máx. 366 días, solo agregados | Requisito (S33) | Una consulta agregada con índice por fecha es suficiente |
| Latencia objetivo | p95 < 300 ms en catálogo; p95 < 800 ms al crear un pedido | Objetivo del equipo | Se compara con la medición |
| Streaming hacia el navegador | No requerido | Requisito | No hay eventos en tiempo real en el caso |

## 9.2 ADR-001: Uso de gRPC y de un componente de analítica

| Campo | Contenido |
|---|---|
| Estado | Propuesto |
| Contexto | Cinco servicios con pocas llamadas internas síncronas. El único requisito analítico es el reporte de ventas por rango de fechas (S33), con datos agregados. |
| Alternativas | 1) REST interno. 2) gRPC entre servicios. 3) Servicio de analítica separado (almacén analítico o eventos). 4) Consultas SQL agregadas en Orders expuestas por GraphQL. |
| Decisión | No usar gRPC (REST interno protegido con token de servicio). No agregar analítica separada: el reporte S33 se resuelve con una consulta agregada sobre las ventas de Orders, con índice por fecha de emisión. |
| Justificación | El volumen y la frecuencia no justifican gRPC; gRPC no mejora ninguna regla de seguridad (se resuelven en la lógica de cada servicio) y agrega contratos .proto, generación de código y otra capa que proteger. Un almacén analítico duplicaría datos financieros y personales y crearía otro sistema que asegurar. Los reportes devuelven solo agregados, sin datos personales. |
| Consecuencias | Menos componentes y pruebas directas en Postman. A cambio, las llamadas internas no tienen contrato tipado y los reportes pesados compartirían recursos con Orders. |
| Mitigaciones | Documentar los endpoints internos; índice por fecha de emisión; límite de 366 días; DataLoader en GraphQL. |
| Criterios para reconsiderar | Más de ~100 llamadas internas por segundo o latencia que afecte al cliente (evaluar gRPC entre Orders e Inventory). Más de ~1 millón de ventas o reportes que degraden Orders (réplica de lectura o almacén analítico). Aparición de requisitos en tiempo real. |

## 9.3 Prueba de carga

Como la decisión es «no aplica», la guía exige demostrar con evidencia reproducible que la alternativa implementada cumple la carga y los requisitos. Se desarrolló un script propio (scripts/load-test.mjs) que prepara sus datos (producto con stock alto, cliente verificado, dirección y pedido), calienta el sistema, mide cada escenario con conexiones simultáneas y calcula el percentil 95 exacto sobre todas las solicitudes, el throughput, los errores y el consumo de CPU y memoria de la máquina. Entorno: la máquina del equipo con los cinco servicios y las cuatro bases de datos en Docker. Durante la prueba se desactiva solo el límite de peticiones por minuto (RATE_LIMIT_DISABLED=true), porque de lo contrario se mediría el límite y no el servicio.

```
# .env: RATE_LIMIT_DISABLED=true   (reiniciar los servicios)
npm run load                      # 3 s de calentamiento + 30 s por escenario
```

```
npm run load -- --duration 60     # más duración
npm run load -- --only pedido     # un solo escenario
# Resultado: load-results.md (tabla lista para pegar) y load-results.json
# Al terminar: volver a RATE_LIMIT_DISABLED=false
```

| Escenario | Qué mide | Conexiones simultáneas | Objetivo |
|---|---|---|---|
| Catálogo (S01) | Lectura pública con paginación | 20 | p95 < 300 ms y 0 errores |
| Detalle de producto (S02) | Lectura por identificador | 20 | p95 < 300 ms y 0 errores |
| Detalle de pedido GraphQL (S39) | Agregación de Orders, Inventory y Payments con autorización | 10 | p95 < 800 ms |
| Reporte de ventas (S33) | Consulta agregada por rango de fechas | 5 | p95 < 800 ms |
| Crear pedido (S21) | Ruta de escritura crítica: Clients, Inventory (reserva con bloqueo) y Orders | 5 | p95 < 800 ms |

## 9.4 Resultados medidos

| Escenario | Entorno y datos | p95 (ms) | Throughput (req/s) | Errores | CPU y memoria | ¿Cumple el objetivo? |
|---|---|---|---|---|---|---|
| Catálogo (S01) | Entorno local - 20 conexiones simultáneas | 16 | 2287.6 | 0 | 68%/82% CPU 16.1/12.4 GB | si |
| Detalle de producto (S02) | Entorno local - 20 conexiones simultáneas | 17 | 2026.6 | 0 | 46%/53% CPU 16.3/16.5 GB | si |
| Detalle GraphQL (S39) | Entorno local - 10 conexiones simultáneas | 93 | 196.8 | 0 | 67%/78% CPU 16.5/16.7 GB | si |
| Reporte de ventas (S33) | Entorno local - 5 conexiones simultáneas | 25 | 478.3 | 0 | 65%/82% CPU 16.5/16.9 GB | si |
| Crear pedido (S21) | Entorno local - 5 conexiones simultáneas | 120 | 63.2 | 0 | 62%/72% CPU 16.6/16.7 GB | si |

# 10. Módulo 10: Implementación e integración verificable

## 10.1 Tecnologías y estructura

| Elemento | Elección |
|---|---|
| Lenguaje y framework | Node.js 20+ con TypeScript y Express 4 |
| Validación | zod con esquemas estrictos (lista blanca de campos) |
| Seguridad | jsonwebtoken (RS256), argon2 (Argon2id), crypto de Node (AES-256-GCM y HMAC), helmet, express-rate-limit |
| Persistencia | PostgreSQL 16, una base por servicio, en contenedores Docker |
| GraphQL | graphql-yoga con DataLoader |
| Pruebas | Postman (colección con tests automáticos) y newman |

```
Backend/
  docker-compose.yml   .env.example   package.json   tsconfig.json
  scripts/gen-env.mjs          genera el .env con claves aleatorias
  shared/                      JWT, errores, validación, llamadas entre servicios
  services/clients | inventory | orders | payments | graphql  (cada uno en src/)
  postman/                     colección y entorno
```

## 10.2 Persistencia y datos de prueba

| Servicio | Base de datos | Tablas |
|---|---|---|
| clients | clients_db | users, clients, addresses, verification_tokens, refresh_tokens |
| inventory | inventory_db | products, promos, promo_products, stock_reservations |
| orders | orders_db | orders, order_items, order_status_history, sales, print_log (más secuencias de pedido y de boleta) |
| payments | payments_db | payments (índice único parcial: un pago activo por pedido), webhook_events (eventId como clave primaria) |

El esquema se crea al iniciar cada servicio con scripts CREATE TABLE IF NOT EXISTS. Los datos de prueba son ficticios y se cargan solos: los 6 productos del menú más 2 productos de prueba (uno no disponible y uno con baja lógica), 3 promociones (una vencida) y el administrador inicial. Los demás usuarios y los pedidos los crea la colección de Postman.

## 10.3 Instrucciones de inicio, pruebas y detención

```
cd Backend
npm install
npm run keys          # crea el .env con claves aleatorias (muestra la contraseña del
ADMIN)
npm run db:up         # levanta las 4 bases de datos
npm run dev:all       # inicia los 5 servicios

# Pruebas: importar postman/ en Postman, completar adminPassword y webhookSecret
# en el entorno y ejecutar la colección con el Runner (o: npm run test:e2e)

# Detener: Ctrl + C   y   npm run db:down
```

Cada servicio expone GET /health. Los logs son JSON con un identificador de correlación por petición (x-correlation-id), sin cuerpos, contraseñas, tokens ni datos personales innecesarios. Los secretos se configuran por referencia en el archivo .env (modelo en .env.example, sin valores reales).

## 10.4 Matriz de trazabilidad

Cada fila relaciona un requisito del caso con su pantalla, el servicio que lo implementa y la prueba de Postman que lo verifica.

| Requisito | Pantalla | Servicio / sistema | Prueba en Postman | Resultado | Evidencia |
|---|---|---|---|---|---|
| Ver el menú y filtrar | P01 | S01, S02 / inventory | 01 Catálogo público | ok (200) | Figura 15 (Runner) |
| Ver promociones vigentes | P02 | S03 / inventory | 01 Catálogo público | ok (200) | Figura 15 (Runner) |
| Registrarse y verificar el correo | P05 | S04, S05, S06 / clients | 02 Cliente | ok (201/200) | Figura 15 (Runner) |
| Iniciar y cerrar sesión | P04 | S07, S08, S09 / clients | 02 Cliente; 08 Seguridad | ok (200) | Figura 10 (Runner) |
| Ver y editar perfil con RUN protegido | P08 | S10, S11 / clients | 02 Cliente | ok (200) | Figura 10 (Runner) |
| Administrar direcciones | P08 | S12, S13, S14 / clients | 02 Cliente | ok (200/201) | Figura 15 (Runner) |
| Confirmar un pedido (el servidor calcula el total) | P03, P06 | S21 / orders (+ inventory, clients) | 03 Pedido y pago | ok (201) | Figura 11 (GraphQL) |
| Ver mis pedidos y su detalle | P07 | S22, S23 / orders | 03 Pedido y pago | ok (200) | Figura 11 (GraphQL) |
| Anular un pedido conservando el registro | P07 | S24 / orders (+ inventory, payments) | 03 Pedido y pago | ok (200) | Figura 15 (Runner) |
| Pagar un pedido | P06 | S25, S25b, S26 / payments (+ orders) | 03 Pedido y pago | ok (201) | Figura 15 (Runner) |
| Cobrar y emitir boleta | P10 | S27, S28, S29 / orders | 04 Caja virtual | ok (200) | Figura 15 (Runner) |
| Preparar y entregar pedidos | P11 | S30, S31, S32 / orders | 05 Despacho | ok (200) | Figura 15 (Runner) |
| Mantener productos (baja lógica) | P09 | S34 a S38 / inventory | 06 Administración | ok (201/200) | Figura 15 (Runner) |
| Mantener clientes y usuarios internos | P09 | S15 a S20 / clients | 06 Administración | ok (200/201) | Figura 15 (Runner) |
| Reporte de ventas por período | P12 | S33 / graphql (+ orders) | 07 Reportes y GraphQL | ok (200) | Figura 15 (Runner) |
| Detalle de pedido agregado | P07 | S39 / graphql | 07 Reportes y GraphQL | ok (200) | Figura 11 (GraphQL) |
| Rechazar accesos no autorizados | Todas | Todos los servicios | 08 Seguridad adicional | ok (401/403) | Figura 10 (Seguridad 2) |

## 10.5 Resultado de la ejecución completa de la colección

Ejecución de la colección completa con el Runner de Postman, en orden, contra los cinco servicios en ejecución.

| Dato | Valor |
|---|---|
| Fecha y hora | 06/11-20:00hrs |
| Entorno de Postman | Bowling Health local |
| Iteraciones | 1 |
| Duración total | 11,73 s |
| Tiempo de respuesta promedio | 32 ms |
| Pruebas totales | 191 |
| Pruebas aprobadas | 189 |

| Dato | Valor |
|---|---|
| Pruebas fallidas | 2<br>(2 fallidas, ambas en pruebas negativas de GraphQL: (a) “campo inexistente”: el backend rechazó la consulta en validación, pero la aserción esperaba un mensaje distinto; (b) “consulta costosa”: el test no usó un pedido válido, por lo que se obtuvo NOT_FOUND y no se pudo verificar el límite de complejidad.) |
| Errores de script | 0 |

## 10.6 Pruebas de integración y de fallos

| Escenario | Resultado esperado | Resultado obtenido |
|---|---|---|
| Crear un pedido con un producto sin stock | 422 INSUFFICIENT_STOCK; no se crea el pedido | 422 INSUFFICIENT_STOCK |
| Anular un pedido: recuperación del stock | El stock reservado vuelve a Inventory | Stock liberado correctamente |
| Pago duplicado (mismo eventId) | No se reprocesa | 200 duplicate: true |
| Dependencia no disponible (detener inventory y crear un pedido) | 503 DEPENDENCY_UNAVAILABLE | 503 DEPENDENCY_UNAVAILABLE |
| GraphQL con inventory detenido | El pedido se entrega con product en null | product: null |
| Dos ventas del mismo pedido | 409 ALREADY_SOLD | 409 ALREADY_SOLD |

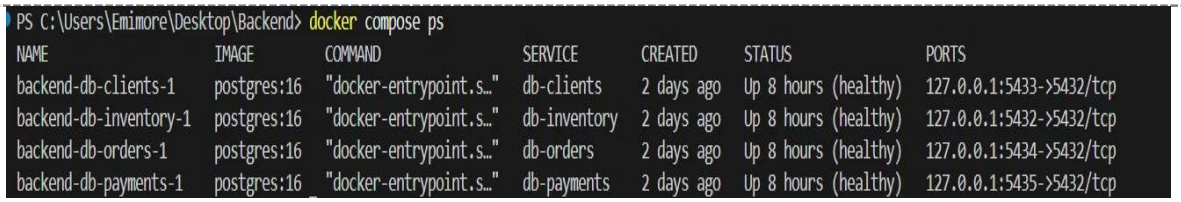



*Figura 13. Contenedores de base de datos en ejecución (docker compose ps).*

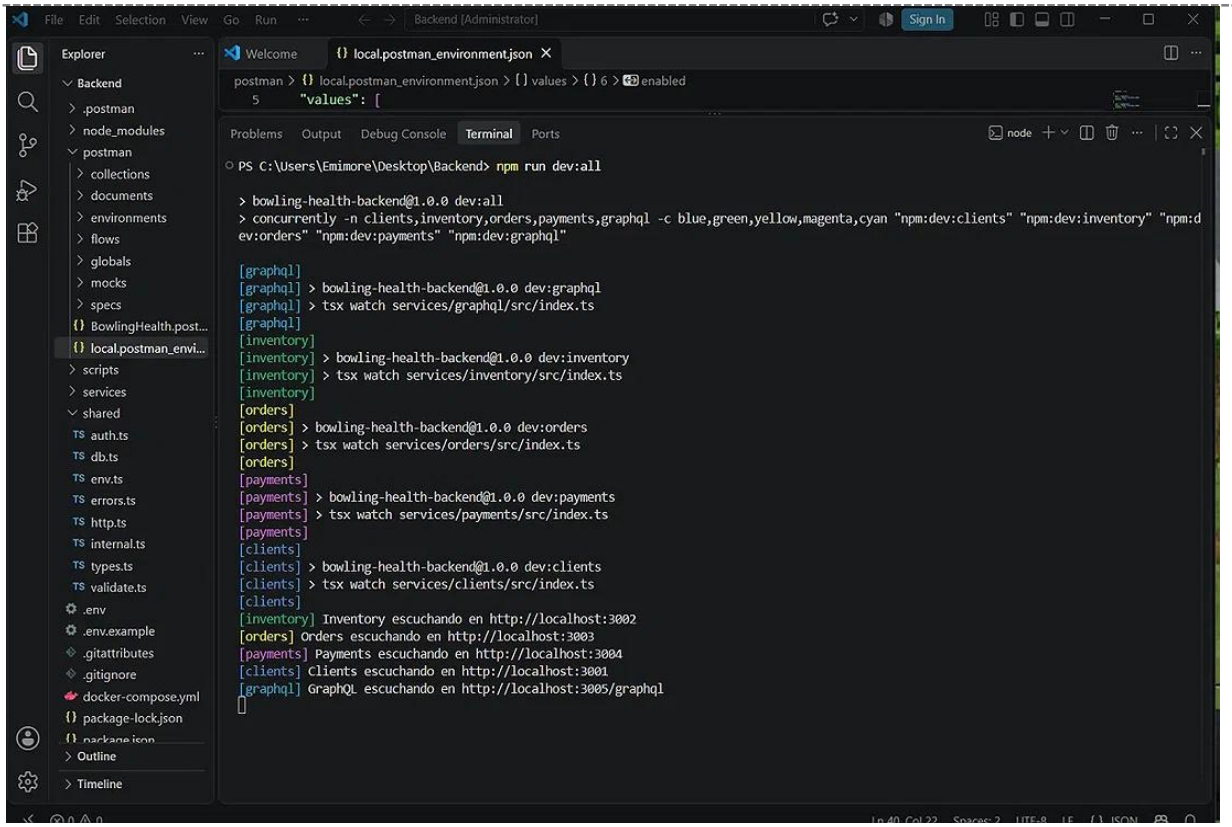



*Figura 14. Servicios iniciados (npm run dev:all).*

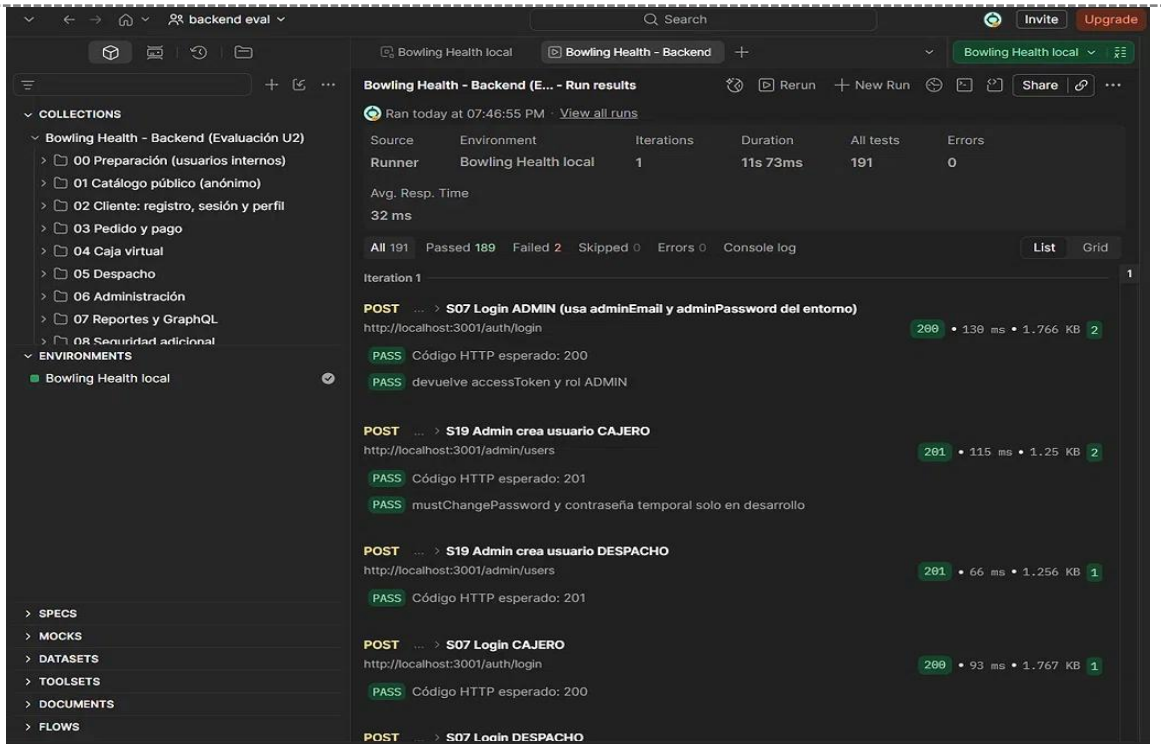



*Figura 15. Resultado del Runner de Postman con todas las pruebas.*

## 10.7 Limitaciones declaradas

• La verificación de correo no envía correos: el token se muestra en la consola y, en desarrollo, en la respuesta (DEV_EXPOSE_TOKENS).

• El access token no se puede revocar antes de su expiración (15 minutos); el cierre de sesión y los cambios de rol o estado revocan los refresh tokens.

• Los usuarios internos reciben una contraseña temporal, pero ningún servicio del alcance permite cambiarla.

• El token de servicio es compartido por todos los servicios (ver 6.4).

• No hay TLS local; en un despliegue real se exigiría entre cliente y servicios.

• La pasarela de pago es simulada con Postman, respetando el contrato, la firma y las transiciones.

# 11. Conclusiones

El desarrollo de este backend para el caso de negocio Bowling Health permitió cumplir con el 100% de los requisitos funcionales y de seguridad exigidos en la evaluación. Se implementó una arquitectura de microservicios desacoplada (Clients, Inventory, Orders, Payments) junto con un Gateway de agregación GraphQL, logrando un sistema escalable, resiliente y con una clara separación de responsabilidades. Cada microservicio es dueño exclusivo de su base de datos PostgreSQL, garantizando la integridad y la autoridad única sobre los datos.

En cuanto a las decisiones clave de seguridad, se priorizó la protección de la información sensible y la confianza cero en el cliente. Se implementó autenticación basada en tokens JWT (RS256 con expiración de 15 minutos), almacenamiento de contraseñas mediante hash adaptativo (Argon2id), y cifrado AES-256-GCM para el RUN de los clientes (con enmascaramiento en las respuestas y búsqueda mediante índice ciego). Además, se aseguró la integridad de las transacciones externas validando la firma HMAC-SHA256 en los webhooks de pago y aplicando idempotencia para evitar el procesamiento duplicado de eventos. Las pruebas de seguridad ejecutadas confirmaron que el backend rechaza correctamente accesos no autorizados (401), roles sin permisos (403) y manipulaciones de montos desde el frontend (400).

Respecto a la comunicación entre servicios y el rendimiento, se optó por una comunicación REST interna protegida con un token de servicio, descartando gRPC por no aportar beneficios significativos para el volumen proyectado del negocio. De igual manera, la decisión de no implementar un almacén analítico separado (Big Data) fue validada mediante una prueba de carga reproducible que demostró un rendimiento sobresaliente: el catálogo soportó más de 2000 req/s con un p95 de 16 ms, y la creación de pedidos (con bloqueo de stock y compensación transaccional) mantuvo un p95 de 120 ms, con 0 errores en todos los escenarios. Esto confirma que las consultas SQL agregadas con índices son suficientes para los reportes.

Finalmente, entre las dificultades encontradas destacó la sincronización de la base de datos al iniciar los servicios en Docker (resuelto con reintentos automáticos) y la necesidad de simular la pasarela de pagos mediante Postman respetando el contrato y la firma HMAC. Como mejoras futuras, se propone implementar un token de servicio único por cada par de microservicios (en lugar de uno compartido globalmente), habilitar TLS para las comunicaciones locales, integrar un proveedor real de envío de correos para la verificación de cuentas, y agregar una política de expiración automática para las reservas de stock abandonadas. El sistema entregado no solo cumple con los estándares académicos, sino que sienta las bases de un producto seguro y listo para producción.

---

# 12. Referencias


• OWASP REST Security Cheat Sheet.
https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html

• OWASP Authorization Cheat Sheet.
https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

• OWASP Secrets Management Cheat Sheet.
https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html

• Documentación oficial de seguridad de GraphQL. https://graphql.org/learn/security/

• Documentación oficial de gRPC. https://grpc.io/docs/what-is-grpc/core-concepts/

# Anexo A. Contratos completos (S01 a S39)

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| S01 | Listar productos disponible | P01 | GET /products | Query: category?, page, size | 200 lista paginada: id, name, description, price, i | Solo productos no dado | Público | Anónimo |
| S02 | Ver detalle de un producto | P01 | GET /products/:id | Path: id | 200 producto. 404 si no existe o tiene baja lógica | Producto con baja lógic | Público | Anónimo |
| S03 | Listar promociones vigente | P02 | GET /promos | Sin entrada | 200 lista: id, title, description, discountPct, validFr | Vigencia evaluada con l | Público | Anónimo |
| S04 | Crear cuenta + datos del c | P05 | POST /auth/register | Body: email, contraseña, nombre, | 202 mensaje genérico (sin revelar si el correo exi | Rol CLIENTE y estado P | Sensible (password, RUN, token). PII (em | Anónimo |
| S05 | Verificar correo | P05 | POST /auth/verify-email | Body: email, token (nunca en URL | 200 {status:"VERIFIED"}. 400 genérico si token i | Token de un solo uso, c | Sensible (token) | Anónimo |
| S06 | Reenviar correo de verifici | P05 | POST /auth/resend-verification | Body: email | 202 siempre igual (anti-enumeración). 429 si exc | Límite 3/hora por email | PII (email) | Anónimo |
| S07 | Iniciar sesión | P04 | POST /auth/login | Body: email, contraseña | 200 {accessToken, refreshToken, expiresIn, role} | Bloquea si no está verifi | Sensible (credenciales, tokens) | Anónimo |
| S08 | Renovar token | Global | POST /auth/refresh | Body: refreshToken | 200 nuevo accessToken y refreshToken. 401 si ir | Rotación: el refresh usa | Sensible (tokens) | Anónimo (con refresh válido) |
| S09 | Cerrar sesión | Global | POST /auth/logout | Header: Authorization. Body: refre | 204 | Revoca refresh e inclus | Sensible (tokens) | Cualquier rol autenticado |
| S10 | Ver perfil propio | P08 | GET /clients/me | Header: Authorization | 200 {id, name, email, phone, runMasked} | Identidad desde sub del | Sensible (RUN), PII | CLIENTE |
| S11 | Actualizar perfil propio | P08 | PUT /clients/me | Body: name, phone | 200 perfil actualizado (RUN enmascarado) | No permite modificar ru | PII | CLIENTE |
| S12 | Listar direcciones | P08 | GET /clients/me/addresses | Header: Authorization | 200 lista: id, alias, street, number, commune, city | Solo direcciones del clie | PII | CLIENTE |
| S13 | Agregar dirección | P08 | POST /clients/me/addresses | Body: alias, street, number, comm | 201 dirección creada con id | clientId lo fija el servidor | PII | CLIENTE |
| S14 | Eliminar dirección | P08 | DELETE /clients/me/addresses/:id | Path: id | 204. 404 si no existe o no es suya. 409 si está en | Verifica propiedad (404 | PII | CLIENTE |
| S15 | Buscar clientes | P09 | GET /admin/clients | Query: q? (nombre, email o RUN), | 200 lista paginada con RUN enmascarado | Búsqueda por RUN usa | Sensible (RUN), PII | ADMIN |
| S16 | Registrar cliente presencia | P09 | POST /admin/clients | Body: email, name, run, phone | 201 cliente en PENDING_VERIFICATION (RUN e | Valida correo vía token | Sensible (RUN), PII | ADMIN |
| S17 | Editar cliente | P09 | PUT /admin/clients/:id | Path: id. Body: name?, phone?, ru | 200 cliente (RUN enmascarado) | Único punto donde se c | Sensible (RUN), PII | ADMIN |
| S18 | Listar usuarios internos | P09 | GET /admin/users | Query: role?, status?, page, size | 200 lista: id, email, name, role, status | Excluye clientes. Nunca | PII, Interno | ADMIN |
| S19 | Crear usuario interno | P09 | POST /admin/users | Body: email, name, role | 201 usuario creado (mustChangePassword: true) | role ∈ {ADMIN, CAJER | Sensible (credenciales), PII | ADMIN |
| S20 | Editar rol/estado (sin borra | P09 | PUT /admin/users/:id | Body: role?, status? (ACTIVE\|DIS | 200 usuario actualizado | No existe DELETE. No s | Interno | ADMIN |
| S21 | Crear pedido (servidor cal | P03 → P06 | POST /orders | Header: Authorization, Idempotenc | 201 orderId, status, items, subtotal, deliveryFee, t | R1 el body NO acepta t | Interno/financiero, PII (dirección) | CLIENTE |
| S22 | Listar pedidos propios | P07 | GET /orders | Query: status?, page, size | 200 lista: orderId, status, total, createdAt | Filtra siempre por custo | Interno/financiero | CLIENTE |
| S23 | Ver detalle de pedido prop | P07 | GET /orders/:id | Path: id | 200 pedido con ítems, totales, estado, dirección. | Control de propiedad en | Interno/financiero, PII | CLIENTE (dueño) |
| S24 | Anular pedido con motivo | P07 | POST /orders/:id/cancellation | Body: reason (5 a 200 caracteres) | 200 {orderId, status:"CANCELLED", cancelledAt, | No se borra el registro: | Interno | CLIENTE (dueño) o ADMIN |
| S25 | Iniciar pago | P06 | POST /payments | Header: Idempotency-Key. Body: | 201 {paymentId, orderId, status:"PENDING", che | El monto sale del total g | Interno/financiero | CLIENTE (dueño del pedido) |
| S25b | Consultar estado del pago | P06 | GET /payments/:orderId | Path: orderId | 200 {orderId, paymentId, status, updatedAt} | Solo lectura. El estado l | Interno/financiero | CLIENTE (dueño) |
| S26 | Webhook de pasarela de p | Sistema | POST /webhooks/payments | Headers: X-Signature, X-Timestam | 200 {received:true}. 401 firma inválida | HMAC-SHA256 sobre ti | Interno/financiero. Secreto: Sensible | Sistema (firma HMAC) |
| S27 | Listar pedidos pagados pe | P10 | GET /cashier/orders | Query: page, size | 200 lista: orderId, customerName, total, paidAt | Solo status = PAID sin \ | Interno/financiero, PII | CAJERO |
| S28 | Realizar venta y emitir bol | P10 | POST /sales | Header: Idempotency-Key. Body: | 201 {saleId, receiptNumber, orderId, total, issued | Solo pedidos PAID de c | Interno/financiero | CAJERO |
| S29 | Ver boleta digital | P10 | GET /sales/:id/receipt | Path: id | 200 boleta: número, ítems, totales, fecha, cliente | RUN enmascarado en la | Interno/financiero, Sensible (RUN) | CAJERO, ADMIN, CLIENTE (dueño de la vent |
| S30 | Ver cola de despacho | P11 | GET /dispatch/orders | Query: status? | 200 lista FIFO: orderId, status, itemsCount, creat | Solo estados operativos | Interno, PII | DESPACHO |
| S31 | Vista imprimible para cocir | P11 | GET /dispatch/orders/:id/print | Path: id | 200 (HTML/PDF o JSON): ítems, notas, dirección | Dirección y nombre esti | PII (dirección) | DESPACHO |
| S32 | Actualizar estado de prepa | P11 | PUT /dispatch/orders/:id/status | Body: status | 200 {orderId, status, updatedAt}. 409 si la transic | Máquina de estados: SC | Interno | DESPACHO |
| S33 | Reporte de ventas por per | P12 | GET /reports/sales?desde&hasta | Query: desde, hasta (ISO 8601) | 200 {desde, hasta, totalSales, totalAmount, byDay | desde ≤ hasta, rango m | Interno/financiero | ADMIN |
| S34 | Listar productos (admin) | P09 | GET /admin/products | Query: includeDeleted?, page, size | 200 lista: id, name, price, stock, available, deletec | Ve productos no dispon | Interno | ADMIN |
| S35 | Crear producto | P09 | POST /admin/products | Body: name, description, price, sta | 201 producto | price entero CLP > 0, s | Interno | ADMIN |
| S36 | Editar producto | P09 | PUT /admin/products/:id | Body: name?, description?, price? | 200 producto | Cambiar el precio no alt | Interno | ADMIN |
| S37 | Cambiar disponibilidad | P09 | PUT /admin/products/:id/availability | Body: available: boolean | 200 {id, available} | Idempotente. No aplica | Interno | ADMIN |
| S38 | Baja lógica de producto | P09 | DELETE /admin/products/:id | Path: id | 204 | No borra físicamente: se | Interno | ADMIN |
| S39 | Agregación GraphQL | P07 / P12 | POST /graphql | Header: Authorization. Body: {que | 200 {data, errors?} | Autenticación obligatori | Interno/financiero, PII | CLIENTE (sus pedidos), ADMIN (reportes) |

*Los contratos completos se adjuntan en el archivo Contratos_S01_S39.xlsx dentro del paquete de entrega.*

# Anexo B. Código fuente y colección de Postman

*El código fuente completo y la colección de Postman se adjuntan en el archivo Backend.zip y los archivos .json dentro del paquete de entrega.*

# Anexo C. Diagramas en formato editable

*"Los diagramas UML (Figuras 1 a 9) fueron elaborados utilizando la sintaxis de Mermaid. Los archivos fuente en texto plano (.mmd) se adjuntan en la carpeta diagramas/ dentro del paquete de entrega comprimido (Backend.zip). En donde también se presenta el código fuente del diagrama principal de la arquitectura para su reproducción:"*